

A market-based efficient matching mechanism for crowdsourced delivery systems with demand/supply elasticities

Yuki Oyama ^a,* , Takashi Akamatsu ^b,*

^a Department of Civil Engineering, The University of Tokyo, Tokyo, Japan

^b Graduate School of Information Sciences, Tohoku University, Miyagi, Japan

ARTICLE INFO

Dataset link: <https://github.com/yuki-oyama/csd-elias>

Keywords:

Crowdsourced delivery
Matching market design
Fluid–particle decomposition
Auction mechanism
Perturbed utility theory
Traffic assignment problem

ABSTRACT

Crowdsourced delivery (CSD), or crowd-shipping, is an emerging business model that leverages the underutilized or excess capacity of individual drivers to fulfill delivery tasks. This paper presents a general formulation of a large-scale two-sided CSD matching problem, considering demand/supply elasticity, heterogeneous preferences of both shippers and drivers, and task-bundling. We propose a set of methodologies to solve this problem. First, we reveal that the fluid–particle decomposition approach of Akamatsu and Oyama (2024) can be extended to our general formulation. This approach decomposes the original large-scale matching problem into a fluidly-approximated task partition problem (master problem) and small-scale particle matching problems (sub-problems). We propose to introduce a truthful and efficient auction mechanism to the sub-problems, which enables the observation of privately perceived costs for each shipper/driver. Furthermore, by finding a theoretical link between the auction problems and random/perturbed utility theory, we succeed in accurately reflecting the information collected from the auctions to the master problem in a theoretically consistent manner. This reduces the master problem to a smooth convex optimization problem, theoretically guaranteeing the computational efficiency and solution accuracy of the fluid approximation. Second, we transform the master problem into a traffic assignment problem (TAP) based on a task-chain network representation. This transformation overcomes the difficulty in enumerating task bundles. Finally, we formulate the dual problem of the TAP, whose decision variable is only a price/reward pattern at market equilibrium, and develop an efficient accelerated gradient descent method. The numerical experiments clarify that the proposed approach drastically reduces the computational cost of the general and complex CSD matching problem (~ 700 times faster than a naive method) without sacrificing the accuracy of the optimal solution (mostly within 0.5% errors).

1. Introduction

The rapid growth of e-commerce and rising customer expectations of fast, low-price, and punctual delivery have significantly increased the demand for last-mile parcel delivery. Last-mile delivery is challenging for traditional couriers with a limited number of drivers due to the large volume of small but frequent delivery requests, necessitating novel and cost-effective delivery solutions. In recent years, the widespread adoption of smart mobile and communication technology has stimulated the development of on-demand mobility services and the gig economy. Crowdsourced delivery (CSD), or crowd-shipping, is an emerging business model

* Corresponding authors.

E-mail addresses: oyama@bin.t.u-tokyo.ac.jp (Y. Oyama), akamatsu@plan.civil.tohoku.ac.jp (T. Akamatsu).

<https://doi.org/10.1016/j.trc.2025.105110>

Received 30 December 2024; Received in revised form 10 March 2025; Accepted 16 March 2025

Available online 1 April 2025

0968-090X/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (<http://creativecommons.org/licenses/by-nc/4.0/>).

for city logistics inspired by this trend. CSD leverages the underutilized or excess capacity of individual drivers, such as commuters, travelers, and shoppers, to fulfill delivery tasks (Le et al., 2019). By utilizing existing trips of ordinary drivers, CSD is expected to not only perform fast and cost-effective deliveries but also reduce the negative environmental impact associated with the use of dedicated delivery vehicles, such as traffic, pollutant emissions, and energy consumption (Mladenow et al., 2016; Paloheimo et al., 2016; Simoni et al., 2020).

In this paper, we study a two-sided CSD matching market of Fig. 1, where the platform acts as an intermediary to match shippers (demand) and drivers (supply). Drivers are ordinary individuals who have their own origin–destination pairs and perform delivery tasks on the way by making detours. Both shippers and drivers decide on their participation in the CSD system based on the prices/rewards offered by the platform, and these decisions vary among individuals. We also consider task-bundling, that is the possibility of drivers performing multiple delivery tasks during a single trip. Although realistic, these settings are associated with the issues of (i) computational complexity, (ii) supply and demand uncertainties, and (iii) heterogeneity in individual preferences, and make the matching problem very challenging in today's e-commerce context with a large demand for parcel delivery. We aim to simultaneously solve all three issues, while they have been only independently addressed in the literature, as explained below.

First, the CSD matching problem is a large-scale combinatorial optimization problem that is generally difficult to solve in a reasonable computational time. This problem becomes even harder when task-bundling is considered, as the number of possible task bundles explodes as that of tasks increases. To reduce computational complexity, some studies decompose the problem into smaller-scale and tractable matching problems, e.g., by consolidating pickup points to a limited number of depots (Wang et al., 2016; Archetti et al., 2016; Arslan et al., 2019) or clustering tasks based on their geographical locations (Huang et al., 2021; Elskokary et al., 2023; Simoni and Winkenbach, 2023). However, a two-stage optimization that relies on arbitrary problem decomposition does not ensure global matching efficiency. It may also generate a partitioned set of delivery tasks with a wide range of distances, particularly in cases where the pickup and delivery locations of requests are widely distributed across the entire network, leading to an inefficient matching pattern. In addition, most existing studies assume one-to-one matching in which only a single task is assigned to each driver (e.g., Archetti et al., 2016; Dayarian and Savelsbergh, 2020; Çınar et al., 2024; Akamatsu and Oyama, 2024), although assigning a bundle of tasks to a driver can significantly reduce the total transportation cost and the compensation to be paid due to the sub-additivity of the costs (Gansterer and Hartl, 2018; Mancini and Gansterer, 2022; Wang et al., 2023). Considering task-bundling adds extra complexity to the matching problem and has never been addressed in a large-scale setting in the literature.

Second, CSD systems often consider ordinary individuals, such as commuters and travelers, as supply resources. These drivers are called occasional/ad-hoc/freelance drivers (Archetti et al., 2016; Arslan et al., 2019; Yang et al., 2024), and they are not dedicated drivers but travel anyway for their purposes. Therefore, the participation of drivers in the CSD system is stochastic and depends on compensation (Le and Ukkusuri, 2019b; Dayarian and Savelsbergh, 2020), i.e., the supply is price-elastic. The decision of drivers not to work for CSD leads to supply uncertainty and can significantly influence matching efficiency and operational costs (Hou et al., 2022; Yang et al., 2024). To deal with supply uncertainty in CSD matching, recent studies model drivers' acceptance behavior of dispatching orders as a function of compensation (Dahle et al., 2019; Le et al., 2021; Barbosa et al., 2023; Çınar et al., 2024; Hou et al., 2022, 2025). Integrating a driver acceptance behavior model, Hou et al. (2022, 2025), and Çınar et al. (2024) design personalized rewards through optimization and achieve a high acceptance rate and cost savings. However, these studies test their approaches only on small-scale problems and assume a one-to-one matching in which only a single task is offered to each driver. Moreover, most studies ignore the impact of reward values on demand, i.e., demand elasticity, despite the fact that increased rewards (salary) may lead to increased shipping fees and thus demand uncertainty.

Finally, the willingness of drivers to work for the CSD and the willingness of shippers to pay for delivery can be very heterogeneous between individuals (Le and Ukkusuri, 2019a; Le et al., 2019; Dayarian and Savelsbergh, 2020). Finding an optimal matching pattern requires information on these heterogeneous preferences, but they are generally unobservable (Akamatsu and Oyama, 2024). This unobserved heterogeneity generates stochasticity of behavior and supply/demand uncertainty. Collecting choice data is a way to address heterogeneity (Le and Ukkusuri, 2019b; Barbosa et al., 2023; Hou et al., 2022), but such data do not necessarily elicit true preferences because the truly preferred alternatives for individuals may not be available/offered in the observed choice situations. In contrast, several previous works study market-based CSD matching with bidding (Kafle et al., 2017; Allahviranloo and Baghestani, 2019; Su et al., 2023; Mancini and Gansterer, 2022). Performing auctions with a *truthful* market mechanism, such as the Vickrey-Clarke-Groves (VCG) mechanism (Vickrey, 1961; Clarke, 1971; Groves, 1973), enables the observation of individual preferences through bids. However, the VCG mechanism is computationally expensive and does not apply to a large-scale problem, and previous studies on bid-based CSD systems do not present mechanisms to elicit true preferences.

In our previous study (Akamatsu and Oyama, 2024), we proposed a fluid-particle decomposition (FPD) approach to deal with a large-scale CSD matching problem with unobserved heterogeneous driver preferences. The approach decomposes the matching problem into task partition (master problem) and individual task-driver matching within smaller groups of drivers (sub-problems). We applied the VCG mechanism to each sub-problem to observe true perceived costs through bids. Moreover, the FPD approach performs task partition so that the global matching efficiency is achieved. To approximate the optimal value functions of the sub-problems without explicitly solving them, we exploit the random utility modeling (RUM) framework and fluidly approximate the master problem. This way, the FPD approach addresses the issues of (i) computational complexity and (ii) heterogeneous preferences and achieves a highly accurate matching pattern with a very efficient computation. However, the previous study solves one-to-one matching and ignores demand and supply elasticities.

In this paper, we study a more general and complex matching problem that takes into account demand and supply elasticities and task-bundling, which are practical requirements for CSD systems. We model a two-sided market: On one side, shippers decide

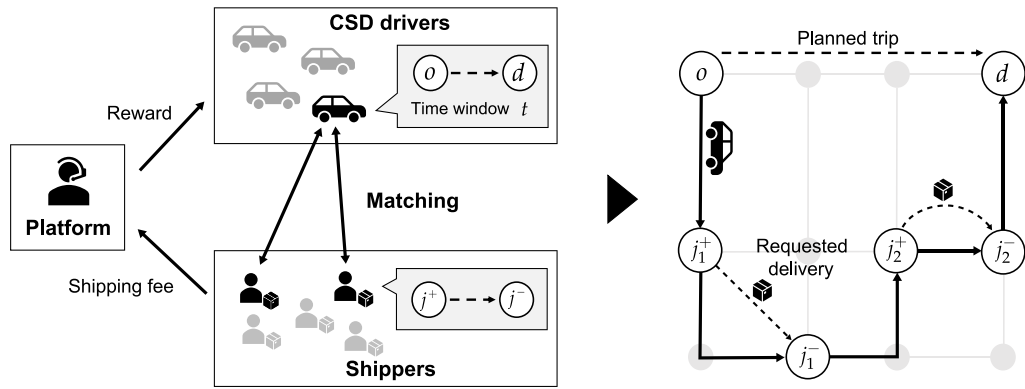


Fig. 1. Conceptual diagram of the CSD system considered in this study.

the time window to request delivery tasks, and on the other side, drivers decide which delivery tasks to perform. We also consider opt-out options so that shippers and drivers can leave the system when prices/rewards do not meet their preferences. In addition, drivers are allowed to perform multiple delivery tasks during a single trip.

To solve this more generalized matching problem, we propose novel and efficient algorithms by extending the FPD approach. Fig. 2 shows a complete picture of the approach of this study. The FPD approach decomposes the original matching problem into a fluidly approximated task partition problem (master problem) and particle matching problems (sub-problems) within driver and shipper subgroups. Instead of finding an individual matching pattern $(\mathbf{m}, \mathbf{n})$ all at once, the master problem decides the number of shipping permits for each shipper group $(\mathbf{y})$ and the number of task bundles for each driver group $(\mathbf{f})$. Furthermore, to overcome the prohibitive complexity of enumerating task bundles, we transform the master problem into a traffic assignment problem (TAP) on a task-chain network, which reduces to finding an edge flow pattern $\mathbf{x}$ instead of $\mathbf{f}$. We further exploit its dual formulation, whose decision variable is only a price/reward pattern $\mathbf{p}$, and develop a fast gradient-based solution algorithm. These decomposition and transformation techniques dramatically reduce the number of decision variables and computational costs. The sub-problems are small enough for us to quickly solve and apply the VCG mechanism that enables the observation of individual preferences through bids. As such, our proposed approach simultaneously addresses the issues of (i) computational complexity, (ii) supply and demand elasticity, and (iii) individual heterogeneity, which is, to the best of our knowledge, the first achievement in the literature.

The notable contributions of this study are summarized as follows:

- (1) **Formulation:** We formulate a socially optimal CSD matching problem, in which we consider demand/supply elasticity, heterogeneous preferences of both shippers and drivers, and task-bundling, and that the number of shippers and drivers can be very large. This formulation is general and simultaneously considers the three issues (i)-(iii) explained above. We also theoretically prove that the optimality condition coincides with the market equilibrium condition. Matching, pricing, and compensation are simultaneously optimized under the market equilibrium.
- (2) **Approach:** We reveal that the FPD approach of Akamatsu and Oyama (2024) can be extended for our generalized formulation. The extended approach hierarchically decomposes the matching problem into a master problem for task partition and sub-problems for particle matching. The sub-problems for shipper/driver groups are small enough to quickly solve and apply a truthful and efficient auction mechanism, which ensures the observation of true perceived costs. The master problem is fluidly approximated and exploits the perturbed utility theory to evaluate the optimal value functions of the sub-problems in a theoretically consistent manner.
- (3) **Algorithms:** For the general and complex CSD matching problem, we develop efficient algorithms without sacrificing the accuracy of the optimal solution. To overcome the difficulty in enumerating task bundles, we introduce a task-chain network representation on which we cast the task partition problem as a TAP problem. Moreover, by exploiting its dual formulation, we develop an efficient algorithm based on the accelerated gradient descent method. The proposed transformation and algorithm drastically reduce the number of decision variables compared to the original problem, and their efficiency is demonstrated through numerical experiments.

2. Matching problem of crowdsourced delivery

This study models a two-sided CSD market in which a CSD platform acts as an intermediary between a set of shippers and a set of drivers aiming to achieve *socially optimal matching*. While most previous studies regard the platform as a profit maximizer, studying socially optimal matching is essential for public entities to appropriately implement policies and regulations for CSD markets to avoid “market failure” (Akamatsu and Oyama, 2024). In this section, we first formulate the socially optimal matching problem and then show that its optimality conditions coincide with market equilibrium. Note that the list of notations that frequently appear in this study is provided in Appendix A.

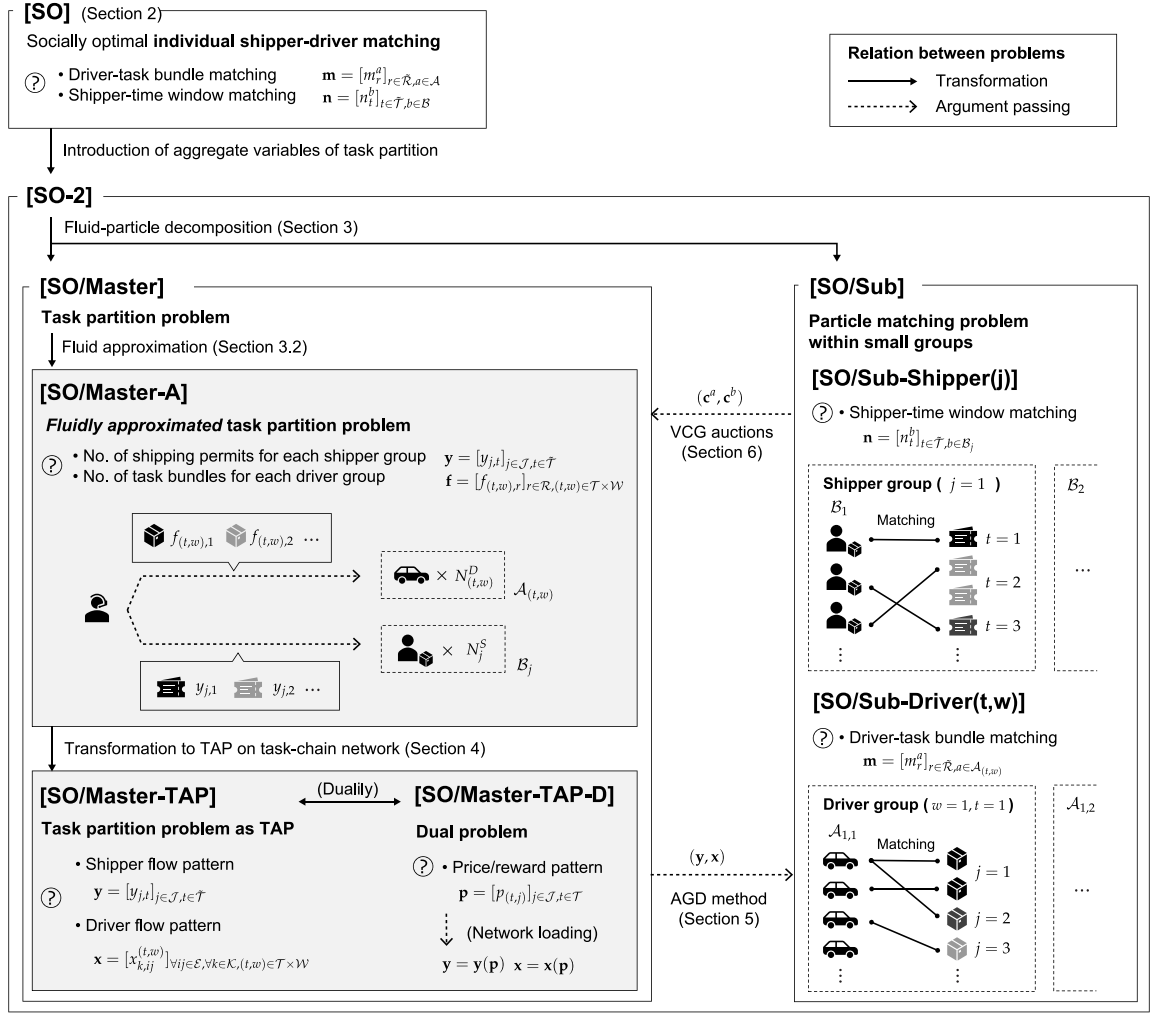


Fig. 2. The full picture of the approach of this study.

2.1. Problem definition and assumptions

Let $\mathcal{G} \equiv (\mathcal{N}, \mathcal{L})$ be a directed graph representing a transportation network where $\mathcal{N}$ and $\mathcal{L}$ are the sets of nodes and links, respectively, and each link $kl \in \mathcal{L}$ is associated with the link travel time t_{kl} . This study assumes that the travel time is static and deterministic and that the travels of CSD participants do not affect it. Also, let $\mathcal{T} \equiv \{1, 2, \dots, T\}$ be the set of time windows. Each time window is assumed to be long enough (e.g., a few hours) for drivers to perform their own trips or delivery tasks. Upon this setting, we define and assume the main components of the CSD system as follows.

- **Delivery task.** A delivery task is a unit of delivery operations and must be delivered from pickup and delivery locations on network $\mathcal{G}$. We assume that tasks are distinguished by only their pickup-delivery location pairs, and denote a task category by $j = (j^+, j^-) \in \mathcal{J} \subseteq \mathcal{N} \times \mathcal{N}$. In other words, all delivery tasks for the same pickup-delivery location pair (j^+, j^-) are homogeneous. We assume each task must be completed within a time window and never done across multiple windows.
- **Platform.** The CSD platform is an intermediary and aims to efficiently match shippers (demand) and drivers (supply). We assume that the platform manager is a non-profitable agent, and the matching objective is the maximization of social surplus. To this end, the platform opens two-sided markets for both shippers and drivers. On one side, the platform issues *shipping permits* for each time window for each group of shippers, and in the market shippers submit bids to obtain the preferred permit. On the other side, the platform assigns the delivery tasks received from shippers to drivers, based on the bids submitted by CSD drivers.
- **Shipper.** A shipper is an agent who has a unit parcel that must be delivered between pickup and delivery points. We assume each shipper has only one delivery task of a single unit, and therefore, we categorize shippers by the task's pickup-delivery pair $j = (j^+, j^-)$. We define the set of shippers $\mathcal{B}_j$ who may request tasks for j and its population $N_j^S = |\mathcal{B}_j|$. Shippers are assumed

to have different preferences for time windows in which their parcels are delivered. As such, a shipper decides whether and in which time window t to ship their parcel. The choice set is $\tilde{\mathcal{T}} = \mathcal{T} \cup \{0\}$ where $t = 0$ represents the opt-out option (i.e., canceling the use of the CSD service).

- **CSD driver.** A CSD driver is an ordinary network traveler who plans to travel for their purpose between an origin–destination (OD) pair $w = (o, d) \in \mathcal{W} \subseteq \mathcal{N} \times \mathcal{N}$ during a time window $t \in \mathcal{T}$, and they can perform delivery tasks on the way to their destination. For instance, an origin and a destination can be a residential zone and a central business district. We define the set of drivers $\mathcal{A}_{(t,w)}$ traveling between OD pair w during time window t and its population $N_{(t,w)}^D = |\mathcal{A}_{(t,w)}|$. We assume that each driver can perform K tasks at maximum but only one task can be executed at a time (i.e., the next task can be executed after the one has been completed). Therefore, each driver's decision is whether and which *task bundle* r to perform considering the detour cost and rewards for the tasks. The choice set is defined as $\tilde{\mathcal{R}} = \mathcal{R} \cup \{\emptyset\}$ where $\mathcal{R}$ is the set of task bundles and $r = \emptyset$ represents not performing any task.

2.2. Behavior of agents

2.2.1. Shipper's behavior

The behavior of a shipper $b \in \mathcal{B}_j$ is to decide whether and in which time window t to ship a parcel for pickup-delivery location pair j , to minimize their perceived cost. Let $n_t^b \in \{0, 1\}$, $\forall t \in \tilde{\mathcal{T}} \equiv \mathcal{T} \cup \{0\}$ be the choice variable of shipper $b \in \mathcal{B}_j$. Here $n_0^b = 1$ indicates that shipper b does not use the CSD service with a perceived cost c_0^b (e.g., representing the cost of requesting a professional courier or delivering the parcel themselves), while $n_t^b = 1$, $\forall t \in \mathcal{T}$ indicates that they ship the parcel in time window t with a perceived cost c_t^b plus the shipping fee $p_{(t,j)}$. Hence, the optimal behavior of the shipper $b \in \mathcal{B}_j$ is represented as the following minimization problem:

$$\min_{n_{b(j)}} n_0^b c_0^b + \sum_{t \in \mathcal{T}} n_t^b (c_t^b + p_{(t,j)}) = \sum_{t \in \tilde{\mathcal{T}}} n_t^b c_t^b + \sum_{t \in \mathcal{T}} n_t^b p_{(t,j)} \quad (2.1)$$

$$\text{subject to } \sum_{t \in \tilde{\mathcal{T}}} n_t^b = 1 \quad (2.2)$$

$$n_t^b \in \{0, 1\} \quad \forall t \in \tilde{\mathcal{T}} \quad (2.3)$$

where the constraint (2.2) represents a single choice of the shipper.

2.2.2. Driver's behavior

In this study, a driver can perform at maximum K tasks during their trip, and we represent a task bundle as a list $r = [j_1, \dots, j_k]$ with $j_{1:k} \in \mathcal{J}$ and $k \leq K$. The behavior of a driver $a \in \mathcal{A}_{(t,w)}$ who travels between OD pair w in time window t is to decide whether or not they perform any delivery task and then choose a task bundle r from the bundle set $\mathcal{R}$ to minimize their perceived detour cost c_r^a minus the reward $p_{(t,r)}$ associated with bundle r , wherein $p_{(t,r)}$ is defined as the sum of the rewards of the tasks included in r . Therefore, the entire choice set is $\tilde{\mathcal{R}} = \mathcal{R} \cup \{\emptyset\}$ where $r = \emptyset$ represents not performing any task.

Let $m_r^a \in \{0, 1\}$, $\forall r \in \tilde{\mathcal{R}}$ be the choice variable of driver $a \in \mathcal{A}_{(t,w)}$, and then their optimal behavior is represented as the following minimization problem:

$$\min_{m_{a(t,w)}} m_{\emptyset}^a c_{\emptyset}^a + \sum_{r \in \mathcal{R}} m_r^a (c_r^a - p_{(t,r)}) = \sum_{r \in \mathcal{R}} m_r^a c_r^a - \sum_{r \in \mathcal{R}} m_r^a \sum_{j \in \mathcal{J}} \delta_{r,j} p_{(t,j)} \quad (2.4)$$

$$\text{subject to } \sum_{r \in \tilde{\mathcal{R}}} m_r^a = 1 \quad (2.5)$$

$$m_r^a \in \{0, 1\} \quad \forall r \in \tilde{\mathcal{R}} \quad (2.6)$$

where $\delta_{r,j} \in \mathbb{Z}_+$ is the number of tasks for j included in r .

2.3. System optimal matching problem

Next, we formulate the socially optimal (SO) matching problem based on the optimization problems mentioned above. To do so, we consider the following demand-supply constraint:

$$\sum_{b \in \mathcal{B}_j} n_t^b \leq \sum_{w \in \mathcal{W}} \sum_{a \in \mathcal{A}_{(t,w)}} \sum_{r \in \mathcal{R}} m_r^a \delta_{r,j} \quad \forall j \in \mathcal{J}, \forall t \in \mathcal{T} \quad (2.7)$$

which ensures that the number of drivers who are willing to deliver tasks j in time window t is equal to or greater than the number of shippers who desire to ship their parcels between j in t .

The social cost to be minimized is the sum of shippers' and drivers' perceived costs (2.1) and (2.4). By the assumption of a nonprofit platform, the following equation representing a money transfer between shippers and drivers holds:

$$\sum_{j \in \mathcal{J}} \sum_{b \in \mathcal{B}_j} \left(\sum_{t \in \mathcal{T}} n_t^b p_{(t,j)} \right) = \sum_{t \in \mathcal{T}} \sum_{w \in \mathcal{W}} \sum_{a \in \mathcal{A}_{(t,w)}} \left(\sum_{r \in \mathcal{R}} m_r^a \sum_{j \in \mathcal{J}} \delta_{r,j} p_{(t,j)} \right) \quad (2.8)$$

which cancels out the money-related terms in (2.1) and (2.4). This means that we only have to consider the perceived costs of shippers and drivers (without the fees/rewards) in the SO matching problem:

$$[\text{SO}] \quad \min_{\mathbf{m}, \mathbf{n}} \quad z(\mathbf{m}, \mathbf{n}) \equiv \sum_{j \in J} \left(\sum_{b \in B_j} \sum_{t \in \tilde{T}} n_t^b c_t^b \right) + \sum_{t \in \tilde{T}} \sum_{w \in \mathcal{W}} \left(\sum_{a \in \mathcal{A}_{(t,w)}} \sum_{r \in \tilde{R}} m_r^a c_r^a \right) \quad (2.9)$$

subject to (2.7),

$$\sum_{t \in \tilde{T}} n_t^b = 1 \quad \forall b \in B_j, \forall j \in J \quad (2.10)$$

$$n_t^b \in \{0, 1\} \quad \forall t \in \tilde{T}, \forall b \in B_j, \forall j \in J \quad (2.11)$$

$$\sum_{r \in \tilde{R}} m_r^a = 1 \quad \forall a \in \mathcal{A}_{(t,w)}, \forall (t, w) \in \mathcal{T} \times \mathcal{W} \quad (2.12)$$

$$m_r^a \in \{0, 1\} \quad \forall r \in \tilde{R}, \forall a \in \mathcal{A}_{(t,w)}, \forall (t, w) \in \mathcal{T} \times \mathcal{W} \quad (2.13)$$

2.4. Optimality conditions and market equilibrium

Due to the 0–1 binary constraints (2.11) and (2.13), the problem [SO] is a combinatorial optimization problem. However, because the constraint matrices are totally unimodular, we can relax the problem [SO] as linear programming by replacing the binary constraints (2.11) and (2.13) with non-negative constraints $n_t^b \geq 0$ and $m_r^a \geq 0$. Based on the linearly relaxed formulation, we can analyze the optimal state of the matching problem [SO] as follows.

We define the Lagrangian $\mathcal{L}$ as

$$\begin{aligned} \mathcal{L}(\mathbf{m}, \mathbf{n}, \boldsymbol{\mu}, \boldsymbol{\rho}, \boldsymbol{\lambda}) := & z(\mathbf{m}, \mathbf{n}) + \sum_{j \in J} \sum_{b \in B_j} \mu_{b(j)} \left(1 - \sum_{t \in \tilde{T}} n_t^b \right) + \sum_{t \in \tilde{T}} \sum_{w \in \mathcal{W}} \sum_{a \in \mathcal{A}_{(t,w)}} \rho_{a(t,w)} \left(1 - \sum_{r \in \tilde{R}} m_r^a \right) \\ & + \sum_{t \in \tilde{T}} \sum_{j \in J} \lambda_{(t,j)} \left(\sum_{b \in B_j} n_t^b - \sum_{w \in \mathcal{W}} \sum_{a \in \mathcal{A}_{(t,w)}} \sum_{r \in \tilde{R}} m_r^a \delta_{r,j} \right) \end{aligned} \quad (2.14)$$

where $\boldsymbol{\mu} \geq 0$, $\boldsymbol{\rho} \geq 0$ and $\boldsymbol{\lambda} \geq 0$ are the Lagrangian multipliers associated with the constraints (2.7), (2.10) and (2.12), respectively. The optimality conditions of [SO] are

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial n_t^b} = c_t^b + \lambda_{(t,j)} - \mu_{b(j)} \geq 0, \quad n_t^b \geq 0, \quad n_t^b \frac{\partial \mathcal{L}}{\partial n_t^b} = 0, \\ \frac{\partial \mathcal{L}}{\partial m_r^a} = c_r^a - \sum_{j \in J} \delta_{r,j} \lambda_{(t,j)} - \rho_{a(t,w)} \geq 0, \quad m_r^a \geq 0, \quad m_r^a \frac{\partial \mathcal{L}}{\partial m_r^a} = 0, \\ \frac{\partial \mathcal{L}}{\partial \lambda_{(t,j)}} = \sum_{b \in B_j} n_t^b - \sum_{w \in \mathcal{W}} \sum_{a \in \mathcal{A}_{(t,w)}} \sum_{r \in \tilde{R}} m_r^a \delta_{r,j} \geq 0, \quad \lambda_{(t,j)} \geq 0, \quad \lambda_{(t,j)} \left(\sum_{b \in B_j} n_t^b - \sum_{w \in \mathcal{W}} \sum_{a \in \mathcal{A}_{(t,w)}} \sum_{r \in \tilde{R}} m_r^a \delta_{r,j} \right) = 0, \\ \frac{\partial \mathcal{L}}{\partial \mu_{b(j)}} = 1 - \sum_{t \in \tilde{T}} n_t^b = 0, \\ \frac{\partial \mathcal{L}}{\partial \rho_{a(t,w)}} = 1 - \sum_{r \in \tilde{R}} m_r^a = 0. \end{cases} \quad (2.15)$$

The conditions (2.15) result in the following three conditions:

$$(\text{Utility maximization of shipper } b \in B_j) \quad \begin{cases} c_t^b + \lambda_{(t,j)}^* = \mu_{b(j)}^* & \text{if } n_t^{b*} > 0 \\ c_t^b + \lambda_{(t,j)}^* \geq \mu_{b(j)}^* & \text{if } n_t^{b*} = 0 \end{cases} \quad \forall t \in \tilde{T} \quad (2.16)$$

$$(\text{Utility maximization of driver } a \in \mathcal{A}_{(t,w)}) \quad \begin{cases} c_r^a - \sum_{j \in J} \delta_{r,j} \lambda_{(t,j)}^* = \rho_{a(t,w)}^* & \text{if } m_r^{a*} > 0 \\ c_r^a - \sum_{j \in J} \delta_{r,j} \lambda_{(t,j)}^* \geq \rho_{a(t,w)}^* & \text{if } m_r^{a*} = 0 \end{cases} \quad \forall r \in \tilde{R} \quad (2.17)$$

$$(\text{Market clearing}) \quad \begin{cases} \sum_{b \in B_j} n_t^{b*} = \sum_{w \in \mathcal{W}} \sum_{a \in \mathcal{A}_{(t,w)}} \sum_{r \in \tilde{R}} m_r^{a*} \delta_{r,j} & \text{if } \lambda_{(t,j)}^* > 0 \\ \sum_{b \in B_j} n_t^{b*} \leq \sum_{w \in \mathcal{W}} \sum_{a \in \mathcal{A}_{(t,w)}} \sum_{r \in \tilde{R}} m_r^{a*} \delta_{r,j} & \text{if } \lambda_{(t,j)}^* = 0 \end{cases} \quad \forall j \in J, \forall t \in \mathcal{T} \quad (2.18)$$

where $\lambda_{(t,j)}^*$ can be interpreted as the reward for a task j in the window t at optimum, i.e., $\lambda_{(t,j)}^* = p_{(t,j)}^*$. That is, the optimal reward $p_{(t,j)}^*$ can be obtained by the Lagrangian multiplier with respect to the constraint (2.7).

The first condition (2.16) describes the cost minimization principle of each shipper. In other words, if $b \in B_j$ chooses t (i.e., $n_t^{b*} = 1$), then their disutility equals the minimum disutility $\mu_{b(j)}^*$. Similarly, the second condition (2.17) is the cost minimization principle of each driver, indicating that the disutility of driver $a \in \mathcal{A}_{(t,w)}$ performing the task bundle r equals the minimum disutility $\rho_{a(t,w)}^*$ when it is chosen (i.e., $m_r^{a*} = 1$). The third market clearing condition (2.18) states that prices are adjusted so as to equilibrate the demand and supply of the delivery tasks. Specifically, at market equilibrium, the number of drivers performing task j in t is equal to the number of shippers who request the delivery of task j in t if the reward $\lambda_{(t,j)}^*$ is larger than zero, and the number of drivers is in excess supply otherwise.

That being said, the optimality conditions of the problem [SO] describe the equilibrium under a perfectly competitive market: (i) the maximization of utility for shippers, (ii) the maximization of utility for CSD drivers, and (iii) the supply and demand equilibrium (market clearing) condition. As such, the following proposition with the definition of equilibrium in the CSD system holds.

Definition 2.1 (*Equilibrium Matching Pattern*). An equilibrium matching pattern under a perfectly competitive market is a tuple of state variables $(\mathbf{m}^*, \mathbf{n}^*, \mu^*, \rho^*, \lambda^*)$ that satisfies the conditions (2.16), (2.17), (2.18).

Proposition 2.1. An equilibrium matching pattern realized under the CSD system maximizes the social surplus defined by (2.9).

3. Fluid–particle decomposition approach

The theoretical analysis in Section 2 reveals that a market equilibrium matching pattern achieves social cost minimization. However, achieving the market equilibrium is non-trivial and requires a mechanism design so that shippers/drivers have no incentive to take strategic action. The platform has to solve [SO] for the mechanism design, but it is challenging due to (i) computational complexity and (ii) the unobservability of the private cost of each shipper/driver. To address these issues, this section presents a hierarchical matching mechanism based on the FPD approach.

3.1. Hierarchical market decomposition

We introduce an aggregate variable $\mathbf{y} = [y_{j,t}]_{j \in \mathcal{J}, t \in \tilde{\mathcal{T}}}$ representing the number of shipping permits for time window t issued for shipper group $\mathcal{B}_j$, as well as $\mathbf{f} = [f_{(t,w),r}]_{r \in \mathcal{R}, (t,w) \in \mathcal{T} \times \mathcal{W}}$ representing the number of task bundles r allocated to driver group $\mathcal{A}_{(t,w)}$.

Then, the problem [SO] can be equivalently formulated as the following integer programming problem:

$$[\text{SO-2}] \quad \min_{\mathbf{m}, \mathbf{n}, \mathbf{y}, \mathbf{f}} \quad \sum_{j \in \mathcal{J}} \left(\sum_{b \in \mathcal{B}_j} \sum_{t \in \tilde{\mathcal{T}}} n_t^b c_t^b \right) + \sum_{t \in \mathcal{T}} \sum_{w \in \mathcal{W}} \left(\sum_{a \in \mathcal{A}_{(t,w)}} \sum_{r \in \mathcal{R}} m_r^a c_r^a \right) \quad (3.1)$$

$$\text{subject to} \quad \sum_{t \in \tilde{\mathcal{T}}} n_t^b = 1 \quad \forall b \in \mathcal{B}_j, \forall j \in \mathcal{J} \quad (2.10)$$

$$\sum_{r \in \mathcal{R}} m_r^a = 1 \quad \forall a \in \mathcal{A}_{(t,w)}, \forall (t,w) \in \mathcal{T} \times \mathcal{W} \quad (2.12)$$

$$\sum_{b \in \mathcal{B}_j} n_t^b = y_{j,t} \quad \forall t \in \tilde{\mathcal{T}}, \forall j \in \mathcal{J} \quad (3.2)$$

$$\sum_{a \in \mathcal{A}_{(t,w)}} m_r^a = f_{(t,w),r} \quad \forall r \in \mathcal{R}, \forall (t,w) \in \mathcal{W} \times \mathcal{T} \quad (3.3)$$

$$y_{j,t} \leq \sum_{w \in \mathcal{W}} \sum_{r \in \mathcal{R}} \delta_{r,j} f_{(t,w),r} \quad \forall j \in \mathcal{J}, \forall t \in \mathcal{T} \quad (3.4)$$

$$y_{j,t} \in \mathbb{Z}_+ \quad \forall t \in \tilde{\mathcal{T}}, \forall j \in \mathcal{J}, \quad (3.5)$$

$$f_{(t,w),r} \in \mathbb{Z}_+ \quad \forall r \in \mathcal{R}, \forall (t,w) \in \mathcal{W} \times \mathcal{T} \quad (3.6)$$

where $\mathbb{Z}_+$ is the set of all positive integers. The original supply–demand constraint (2.7) is replaced with constraints (3.2)–(3.4) using the intermediate variable $\mathbf{y}$ and $\mathbf{f}$. Note that (2.10) and (3.2), and (2.12) and (3.3) respectively ensure

$$\sum_{t \in \tilde{\mathcal{T}}} y_{j,t} = N_j^S \quad \forall j \in \mathcal{J} \quad (3.7)$$

$$\sum_{r \in \mathcal{R}} f_{(t,w),r} = N_{(t,w)}^D \quad \forall (t,w) \in \mathcal{T} \times \mathcal{W} \quad (3.8)$$

where N_j^S is the size of shipper group $\mathcal{B}_j$ and $N_{(t,w)}^D$ is the size of driver group $\mathcal{A}_{(t,w)}$.

Then, we can hierarchically decompose the problem [SO-2] into a master problem and sub-problems as follows:

[SO/Master]

$$\min_{\mathbf{y}, \mathbf{f}} \quad \sum_{j \in \mathcal{J}} z_j^{S*}(\mathbf{y}_j) + \sum_{t \in \mathcal{T}} \sum_{w \in \mathcal{W}} z_{(t,w)}^{D*}(\mathbf{f}_{(t,w)}) \quad (3.9)$$

subject to (3.4), (3.7), (3.8)

$$\text{where} \quad z_j^{S*}(\mathbf{y}_j) \equiv \min_{\mathbf{n}_{(j)}} z_j^S(\mathbf{n}_{(j)} | \mathbf{y}_j) \quad (3.10)$$

$$z_{(t,w)}^{D*}(\mathbf{f}_{(t,w)}) \equiv \min_{\mathbf{m}_{(t,w)}} z_{(t,w)}^D(\mathbf{m}_{(t,w)} | \mathbf{f}_{(t,w)}) \quad (3.11)$$

[SO/Sub-Shipper(j)]

$$\min_{\mathbf{n}_{(j)}} \quad z_j^S(\mathbf{n}_{(j)} | \mathbf{y}_j) \equiv \sum_{b \in \mathcal{B}_j} \sum_{t \in \tilde{\mathcal{T}}} n_t^b c_t^b \quad (3.12)$$

$$\text{subject to} \quad \sum_{t \in \tilde{\mathcal{T}}} n_t^b = 1 \quad \forall b \in \mathcal{B}_j \quad (3.13)$$

$$\sum_{b \in \mathcal{B}_j} n_t^b = y_{j,t} \quad \forall t \in \tilde{\mathcal{T}} \quad (3.14)$$

$$n_t^b \in \{0, 1\} \quad \forall b \in B_j, \forall t \in \tilde{\mathcal{T}}. \quad (3.15)$$

[SO/Sub-Driver(t, w)]

$$\min_{\mathbf{m}_{(t,w)}} z_{(t,w)}^D(\mathbf{m}_{(t,w)} | \mathbf{f}_{(t,w)}) \equiv \sum_{a \in \mathcal{A}_{(t,w)}} \sum_{r \in \tilde{\mathcal{R}}} m_r^a c_r^a \quad (3.16)$$

$$\text{subject to } \sum_{r \in \tilde{\mathcal{R}}} m_r^a = 1 \quad \forall a \in \mathcal{A}_{(t,w)} \quad (3.17)$$

$$\sum_{a \in \mathcal{A}_{(t,w)}} m_r^a = f_{(t,w),r} \quad \forall r \in \tilde{\mathcal{R}} \quad (3.18)$$

$$m_r^a \in \{0, 1\} \quad \forall r \in \tilde{\mathcal{R}}, \forall a \in \mathcal{A}_{(t,w)} \quad (3.19)$$

The objective function of the master problem (3.9) is defined as the sum of optimal value functions of [SO/Sub-Shipper(j)] and [SO/Sub-Driver(t, w)], respectively represented by (3.12) and (3.16).

3.2. Fluid approximation of the master problem

We now have a master problem and small-scale sub-problems by decomposing the original problem [SO]. The master problem of the proposed mechanism determines (i) the number of shipping permits $\mathbf{y}$ to be issued for each shipper group, and (ii) the number of task bundles $\mathbf{f}$ to be allocated to each driver group. The objective (3.9) of the master consists of the optimal value functions of the sub-problems z^{S*} and z^{D*} , which are obtained as a result of the sub-problems. On the other hand, because the sub-problems are formulated with the quantities $\mathbf{y}$ and $\mathbf{f}$ determined by the master problem as a given, the master problem must be solved before the sub-problems. That is, we in theory must solve the master problem and sub-problems simultaneously.

To address this issue, we extend the fluid approximation of Akamatsu and Oyama (2024). The core idea of the approach is evaluating z^{S*} and z^{D*} as closed-form functions of $\mathbf{y}$ and $\mathbf{f}$ by considering the continuous distributions of shippers' and drivers' utilities. The rationale behind this is that if the auctions described in Section 6 are conducted on a daily basis, the platform manager can collect the information on shippers' and drivers' utilities sufficiently to estimate the distribution, even though the utilities of the shippers and drivers appearing on the day are still unknown. Specifically, we rely on an ARUM framework as follows.

Assumption 3.1. There are sufficiently large numbers of shippers for each $j \in \mathcal{J}$ and drivers for each $(t, w) \in \mathcal{T} \times \mathcal{W}$. Then, their perceived costs c_t^b and $c_{(t,w),r}^a$ can be approximated by

$$c_t^b \approx C_{j,t}^S - \zeta_{j,t}^b \quad \forall b \in B_j, \forall j \in \mathcal{J}, \forall t \in \tilde{\mathcal{T}}, \quad (3.20)$$

$$c_{(t,w),r}^a \approx C_{(t,w),r}^D - \epsilon_{(t,w),r}^a \quad \forall a \in \mathcal{A}_{(t,w)}, \forall (t, w) \in \mathcal{W} \times \mathcal{T}, \forall r \in \tilde{\mathcal{R}} \quad (3.21)$$

where $C_{j,t}^S$ is the deterministic cost of a shipper shipping their parcel in time window $t \in \mathcal{T}$ or not using the CSD service ($t = 0$), and $C_{(t,w),r}^D$ is the deterministic detour cost for a driver who travels between OD pair w in time window t performing a task bundle $r \in \tilde{\mathcal{R}}$ or not participating in the system ($r = 0$). $\zeta_{j,t}^b$ and $\epsilon_{(t,w),r}^a$ are the unobservable random utilities following joint distributions with finite means that are continuous and independent of $C_{j,t}^S$ and $C_{(t,w),r}^D$, respectively.

Typically, the deterministic cost $C_{(t,w),r}^D$ of a driver performing a task bundle r is defined as the detour travel time (e.g., Wang et al., 2016; Akamatsu and Oyama, 2024), while that cost $C_{j,0}^S$ for a shipper not using the CSD service can represent the cost for requesting a professional delivery service or shipping the parcel themselves.

Under Assumption 3.1, the negative surplus functions of shippers and drivers are obtained by the following *expected minimum cost* functions:

$$\Pi_j^S(\mathbf{v}_j^S) = \mathbb{E} \left[\min_{t \in \tilde{\mathcal{T}}} \{v_{j,t}^S - \zeta_{j,t}^b\} \right], \quad (3.22)$$

$$\Pi_{(t,w)}^D(\mathbf{v}_{(t,w)}^D) = \mathbb{E} \left[\min_{r \in \tilde{\mathcal{R}}} \{v_{(t,w),r}^D - \epsilon_{(t,w),r}^a\} \right], \quad (3.23)$$

where $v_{j,t}^S = C_{j,t}^S + p_{(t,j)}$ and $v_{(t,w),r}^D = C_{(t,w),r}^D - \sum_{j \in \mathcal{J}} \delta_{r,j} p_{(t,j)}$ are the generalized deterministic costs. Moreover, for any joint distribution of random utilities $\zeta_{j,t}^b$ and $\epsilon_{(t,w),r}^a$, we can find the following *perturbation functions*, or generalized entropy functions (Sørensen and Fosgerau, 2022; Fosgerau et al., 2020), as the convex conjugate of the surplus functions Π_j^S and $\Pi_{(t,w)}^D$ (see, for example Rockafellar, 1970):

$$\mathcal{H}_j^S(\mathbf{P}_j^S) = \min_{\mathbf{v}_j^S} \left\{ \mathbf{v}_j^{S\top} \mathbf{P}_j^S - \Pi_j^S(\mathbf{v}_j^S) \right\}, \quad (3.24)$$

$$\mathcal{H}_{(t,w)}^D(\mathbf{P}_{(t,w)}^D) = \min_{\mathbf{v}_{(t,w)}^D} \left\{ \mathbf{v}_{(t,w)}^{D\top} \mathbf{P}_{(t,w)}^D - \Pi_{(t,w)}^D(\mathbf{v}_{(t,w)}^D) \right\}, \quad (3.25)$$

where $\mathbf{P}_j^S = [P_j^S(t)]_{t \in \tilde{\mathcal{T}}}$ and $\mathbf{P}_{(t,w)}^D = [P_{(t,w)}^D(r)]_{r \in \tilde{\mathcal{R}}}$ are probability distributions according to $\zeta_{j,t}^b$ and $\epsilon_{(t,w),r}^a$, and they can be interpreted as task partition ratios $P_j^S(t) = y_{j,t}/N_j^S$ and $P_{(t,w)}^D(r) = f_{(t,w),r}/N_{(t,w)}^D$. Thus, we obtain

$$\hat{\mathcal{H}}_j^S(\mathbf{y}_j) = N_j^S \mathcal{H}_j^S(\mathbf{P}_j^S), \quad (3.26)$$

$$\hat{H}_{(t,w)}^D(\mathbf{f}_{(t,w)}) = N_{(t,w)}^D \mathcal{H}_{(t,w)}^D(\mathbf{P}_{(t,w)}^D). \quad (3.27)$$

Then, the following proposition holds.

Proposition 3.2. Under [Assumption 3.1](#), the master problem [SO/Master] can be fluidly approximated as

$$\begin{aligned} & \text{[SO/Master-A]} \\ & \min_{\mathbf{y}, \mathbf{f}} \sum_{j \in \mathcal{J}} (\mathbf{C}_j^S \cdot \mathbf{y}_j - \hat{H}_j^S(\mathbf{y}_j)) + \sum_{i \in \mathcal{I}} \sum_{w \in \mathcal{W}} (\mathbf{C}_{(i,w)}^D \cdot \mathbf{f}_{(i,w)} - \hat{H}_{(i,w)}^D(\mathbf{f}_{(i,w)})) \\ & \text{subject to } (3.4), (3.7), (3.8) \end{aligned} \quad (3.28)$$

Proof. [Appendix B](#). $\square$

The problem [SO/Master-A] is a minimization problem in the perturbed utility form ([Fudenberg et al., 2015](#); [Fosgerau et al., 2022](#); [Oyama et al., 2022](#)). As such, the objective function (3.28) represents the total cost under the random utility-maximizing behavior of shippers and drivers. The perturbed functions $\hat{H}^S$ and $\hat{H}^D$ capture the heterogeneity in shippers' and drivers' perceived costs and improve the matching efficiency. This is an important difference from previous studies that consider only deterministic costs (e.g., [Wang et al., 2016](#); [Archetti et al., 2016](#); [Wang et al., 2023](#)).¹

Furthermore, using the convex conjugate duality of $\mathcal{H}$ and Π we can formulate the dual problem of [SO/Master-A].

Proposition 3.3. The Lagrangian dual problem of [SO/Master-A] is given as:

$$\begin{aligned} & \text{[SO/Master-D]} \\ & \max_{\mathbf{p}} G(\mathbf{p}) \equiv \sum_{j \in \mathcal{J}} N_j^S \Pi_j^S(\mathbf{p}) + \sum_{i \in \mathcal{I}} \sum_{w \in \mathcal{W}} N_{(i,w)}^D \Pi_{(i,w)}^D(\mathbf{p}) \end{aligned} \quad (3.29)$$

Proof. [Appendix C](#). $\square$

The dual problem [SO/Master-D] is an unconstrained optimization problem whose objective is the total negative surplus (social cost) of shippers and drivers. The only decision variable is the price vector $\mathbf{p} = [p_{(t,j)}]_{j \in \mathcal{J}, t \in \mathcal{T}}$, which is the dual variable associated with constraint (3.4).

4. Transformation to traffic assignment problem

The fluidly approximated master problem [SO/Master-A] determines continuous variables instead of integer variables, which significantly reduces the computational effort to solve it. However, we still need the enumeration of task bundles $\mathcal{R}_{(t,w)}$ to perform the task-driver matching. Because the enumeration is prohibitively costly when the number of tasks is large, we herein present an efficient solution algorithm for the master problem, in which we cast the task partition problem as a TAP.

4.1. Matching as network path choice

We introduce a *task-chain network* of [Fig. 3](#). This network makes K copies of the task set $\tilde{\mathcal{J}} \equiv \mathcal{J} \cup \{d\}$ and connects each task pair by edge $(i, j) \in \mathcal{E} = \tilde{\mathcal{J}} \times \tilde{\mathcal{J}}$. Then, any bundle of tasks can be represented as a path on the network $r = [j_0, j_1, \dots, j_K, j_{K+1}]$, where $j_0 = o$, $j_{K+1} = d$, and $j_k \in \tilde{\mathcal{J}}, \forall k \in \{1, \dots, K\}$. When $j_k = d$, the driver finishes their trip after completing $k-1$ tasks, and $j_k = j_{k+1} = \dots = j_K = d$. The driver's original trip plan without performing a task ($r = \emptyset$) is represented by the path with $j_1 = d$. As such, the task bundle-driver matching becomes a path choice problem under this network representation.

However, the number of possible paths $|\tilde{\mathcal{R}}|$ is about the size of $(|\mathcal{J}| + 1)^{K+1}$, which is generally very large and its enumeration is computationally prohibitive. To efficiently model the path choice, we exploit the edge-based approach based on the Markov decision process (MDP). That is, the path choice is represented as a recursive decision process of sequential edge choices for $K+1$ stages. To do so, we consider *edge-additive* costs.

Assumption 4.2. The perceived cost of drivers to perform task bundle r is *edge-additive*, that is,

$$C_{(t,w),r}^D + \epsilon_{(t,w),r} = \sum_{k=0}^K \sum_{(i,j) \in \mathcal{E}} \delta_{k,ij}^r (c_{ij}^D + \epsilon_{ij}^{(t,w)}) \quad (4.1)$$

where c_{ij}^D is the edge cost of $(i, j) \in \mathcal{E}$ and ϵ_{ij} is its random term, both of which are independent of k , and $\delta_{k,ij}^r$ equals one if path r uses edge (i, j) at k , i.e., the k th and $(k+1)$ -th tasks of bundle r are i and j , and zero otherwise.

¹ In [Section 8](#), we benchmark our mechanism against a mechanism considering only the deterministic terms of (3.28), which we call *NO mechanism*.

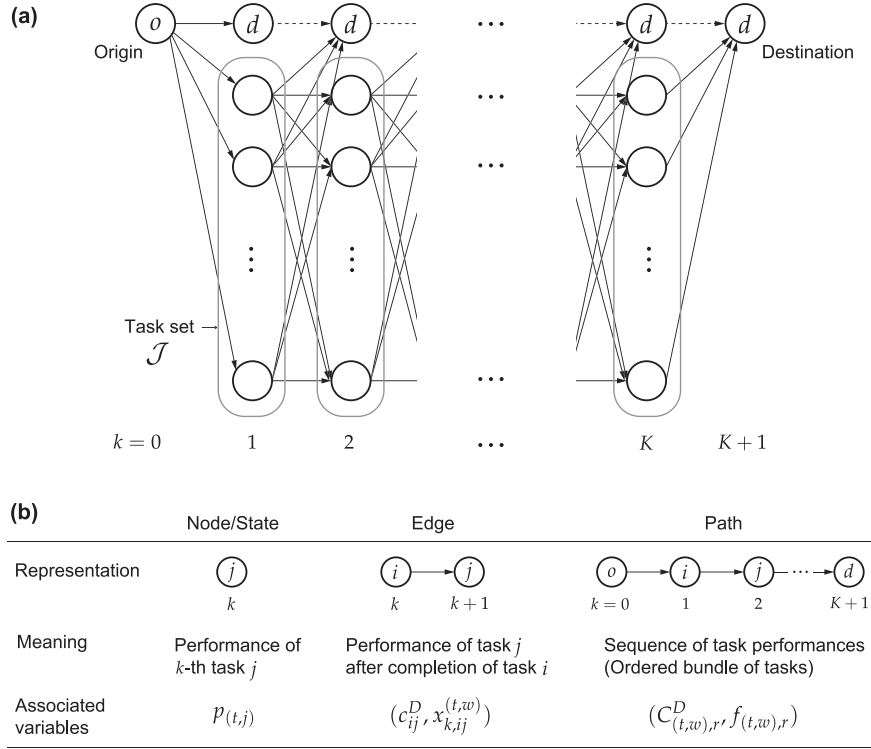


Fig. 3. Task-chain network: (a) network structure and (b) components.

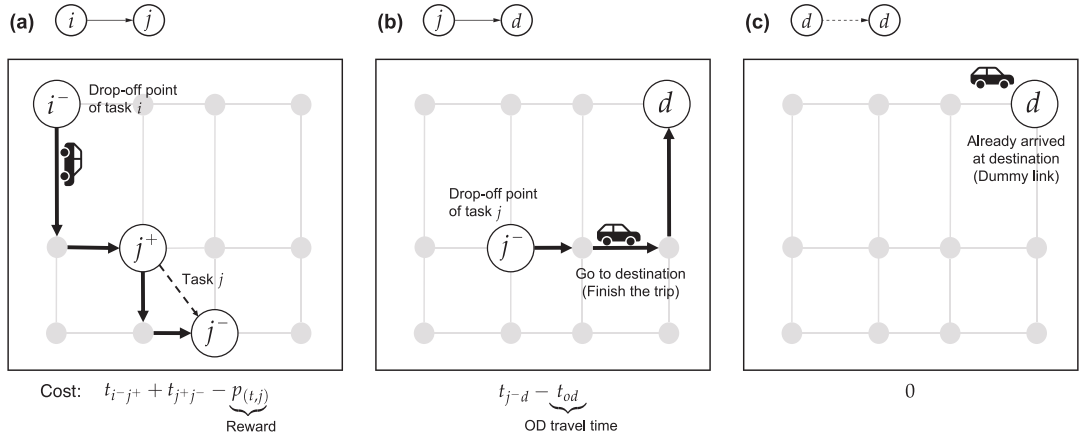


Fig. 4. Three types of edges in the network and the associated costs.

Fig. 4 shows the generalized edge costs on the network. For a pair of two tasks $i, j \in \mathcal{J}$, the cost is equal to the travel cost t_{i-j+} between the drop-off point i^- of task i and the pickup point j^+ of task j , and t_{j+-} between the pickup and drop-off points (j^+, j^-) of task j , minus the reward $p_{(t,j)}$ associated with task j . For edge (j, d) connecting $j \in \mathcal{J}$ to destination d , we subtract the travel cost t_{od} between (o, d) so that the total path cost describes the detour cost for drivers. The network includes dummy edges (d, d) to represent bundles of less than K tasks, and the cost of a dummy edge is zero and deterministic.

Under MDP, the cost-minimizing behavior of drivers is sequentially described at each choice stage k . Then, the path choice probability $P_{(t,w)}^D(r)$ is decomposed into

$$P_{(t,w)}^D(r) = \prod_{k=0}^K \prod_{(i,j) \in \mathcal{E}} P_k^D(j|i)^{\delta_{k,ij}^r} \quad (4.2)$$

where $P_k^D(j|i)$ is the edge choice probability of (i, j) conditional on state (k, i) . In addition, the path-based surplus function $\Pi_{(i,w)}^D$ can be decomposed into $\pi_{(i,w)}$, which is recursively formulated for each state (k, i) as

$$\pi_{(k,i)}^{(t,w)}(\mathbf{p}) = \mathbb{E} \left[\min_{j \in \tilde{\mathcal{J}}} \{c_{ij}^D - p_{(i,j)} - \epsilon_{ij}^{(t,w)} + \pi_{(k+1,j)}^{(t,w)}(\mathbf{p})\} \right]. \quad (4.3)$$

This is also called the value function of each state and represents the shortest path marginal cost from state (k, i) to the target state $(K+1, d)$ on the network (Oyama and Hato, 2019; Oyama, 2023).

4.2. Markovian traffic assignment

Under the network representation, our problem becomes finding the path flow pattern $\mathbf{f}_{(t,w)}$. Moreover, as we exploit an edge-based traffic assignment framework, the decision variable is reduced to the edge flow pattern $\mathbf{x}_{(t,w)}$ that satisfies:

$$\mathbf{x}_{k,ij}^{(t,w)} = \sum_{r \in \mathcal{R}} \delta_{k,ij}^r f_{(t,w),r}. \quad (4.4)$$

At each state (k, i) , the conditional edge choice probability $P_k^D(j|i)$ partitions the sum of upstream flows into edge flows:

$$\mathbf{x}_{k,ij}^{(t,w)} = P_k^D(j|i) \sum_{h \in \tilde{\mathcal{J}}} \mathbf{x}_{k,hi}^{(t,w)}, \quad (4.5)$$

and in particular, at the source state $(0, o)$,

$$\mathbf{x}_{0,oj}^{(t,w)} = P_0(j|o) N_{(t,w)}^D. \quad (4.6)$$

With this vector of edge flows and under the edge-additive assumption of random utilities, the perturbation function $\hat{H}_{(t,w)}^D(\mathbf{f}_{(t,w)})$ is also redefined as

$$\hat{H}_{(t,w)}^D(\mathbf{f}_{(t,w)}) = \bar{H}_{(t,w)}^D(\mathbf{x}_{(t,w)}) = \sum_{k=0}^K \sum_{i \in \tilde{\mathcal{J}}} \bar{H}_{k,i}^{(t,w)}(\mathbf{x}_{k,i}^{(t,w)}) \quad (4.7)$$

where $\mathbf{x}_{k,i}^{(t,w)} = [\mathbf{x}_{k,ij}^{(t,w)}]_{\forall j: (i,j) \in \mathcal{E}}$.

Using this network representation, the problem [SO/Master-A] can be equivalently formulated as the following flow-independent traffic assignment problem (TAP):

[SO/Master-TAP]

$$\min_{\mathbf{y}, \mathbf{x}} \sum_{j \in \mathcal{J}} (\mathbf{C}_j^S \cdot \mathbf{y}_j - \hat{H}_j^S(\mathbf{y}_j)) + \sum_{t \in \mathcal{T}} \sum_{w \in \mathcal{W}} \left(\sum_{k=0}^K \mathbf{c}^D \cdot \mathbf{x}_{k,i}^{(t,w)} - \bar{H}_{(t,w)}^D(\mathbf{x}_{(t,w)}) \right) \quad (4.8)$$

$$\text{subject to } \sum_{t \in \mathcal{T}} y_{j,t} = N_j^S \quad \forall j \in \mathcal{J} \quad (3.7)$$

$$\sum_{i \in \tilde{\mathcal{J}}} \mathbf{x}_{k-1,ij}^{(t,w)} - \sum_{l \in \tilde{\mathcal{J}}} \mathbf{x}_{k,jl}^{(t,w)} = \eta_{k,j} N_{(t,w)}^D \quad \forall k \in \{0, \dots, K+1\}, \forall j \in \mathcal{J} \quad (4.9)$$

$$y_{j,t} \leq \sum_{w \in \mathcal{W}} \sum_{k=0}^K \sum_{i \in \tilde{\mathcal{J}}} \mathbf{x}_{k,ij}^{(t,w)} \quad \forall j \in \mathcal{J}, \forall t \in \mathcal{T} \quad (4.10)$$

where constraint (4.9) is the flow conservation law on the network, and $\eta = [\eta_{k,j}]$ is a two-dimensional vector whose elements equal zero except for $\eta_{K+1,d} = 1$ and $\eta_{0,o} = -1$. This replaces constraint (3.8) of [SO/Master-A]. Constraint (4.10) is the demand-supply constraint ensuring that the number of shipping permits $y_{j,t}$ of time window t for shippers with a task j is less than the number of drivers who perform tasks for j in t , which replaces (3.4) of [SO/Master-A]. The problem [SO/Master-TAP] is a flow-independent and capacity-constrained class of Markovian, or link-based stochastic, traffic assignment problem (Akamatsu, 1996, 1997; Baillon and Cominetti, 2008; Oyama et al., 2022; Akamatsu et al., 2023). More generally, it can be viewed as a perturbed utility traffic assignment problem (Fosgerau et al., 2022; Yao et al., 2024).

Furthermore, the dual problem of [SO/Master-TAP] is obtained by re-formulating [SO/Master-D] as:

[SO/Master-TAP-D]

$$\max_{\mathbf{p}} G(\mathbf{p}) \equiv \sum_{j \in \mathcal{J}} N_j^S \Pi_j^S(\mathbf{p}) + \sum_{t \in \mathcal{T}} \sum_{w \in \mathcal{W}} N_{(t,w)}^D \pi_{(0,o)}^{(t,w)}(\mathbf{p}) \quad (4.11)$$

wherein the driver surplus function $\Pi_{(t,w)}^D$ in the objective (3.29) of [SO/Master-D] is replaced by $\pi_{(0,o)}^{(t,w)}$. As defined in (4.3), the value function $\pi_{(k,j)}^{(t,w)}$ of each state is recursively formulated, and at the source state, $\pi_{(0,o)}^{(t,w)}$ implicitly represents the expected minimum cost among all possible paths on the network, i.e., all task bundles, thus being equivalent to $\Pi_{(t,w)}^D$. This recursively-formulated $\pi_{(k,j)}^{(t,w)}$ can be solved through a dynamic programming approach for any ARUMs (Oyama et al., 2022).

4.3. Examples

4.3.1. Example 1: The multinomial logit model

We herein show as an example the specific case of the multinomial logit (MNL) model for the fluid approximation. To do so, we assume the unobserved costs $\zeta_{j,t}$ and $\epsilon_{ij}^{(t,w)}$ of shippers and drivers follow type I extreme value (EV) distributions: $\zeta_{j,t} \stackrel{iid}{\sim} \text{EV}(0, \theta)$ and $\epsilon_{ij}^{(t,w)} \stackrel{iid}{\sim} \text{EV}(0, \phi)$.

For the MNL models, it is well-known that the perturbation functions are defined as Shannon's entropy functions:

$$\hat{H}_j^S(\mathbf{y}_j) = -\frac{1}{\theta} \sum_{t \in \mathcal{T}} y_{j,t} \ln \frac{y_{j,t}}{N_j^S} \quad (4.12)$$

$$\bar{H}_{(t,w)}^D(\mathbf{x}_{(t,w)}) = \sum_{k=0}^K \sum_{i \in \mathcal{J}} \hat{H}_{k,i}^{(t,w)}(\mathbf{x}_{k,i}^{(t,w)}) = -\frac{1}{\phi} \sum_{k=0}^K \sum_{i \in \mathcal{J}} \sum_{j \in \mathcal{J}} x_{k,i,j}^{(t,w)} \ln \frac{x_{k,i,j}^{(t,w)}}{\sum_{h \in \mathcal{J}} x_{k-1,h,i}^{(t,w)}} \quad (4.13)$$

and accordingly, the surplus functions are the following log-sum functions:

$$\Pi_j^S(\mathbf{p}) = -\frac{1}{\theta} \ln \sum_{t \in \mathcal{T}} \exp\{-\theta(C_{j,t}^S + p_{(t,j)})\} \quad (4.14)$$

$$\pi_{(k,i)}^{(t,w)}(\mathbf{p}) = -\frac{1}{\phi} \ln \sum_{j \in \mathcal{J}} \exp\{-\phi(c_{ij}^D - p_{(t,j)} + \pi_{(k+1,j)}^{(t,w)}(\mathbf{p}))\} \quad (4.15)$$

The recurrence relation of (4.15) reduces to the system of linear equations (Akamatsu, 1996; Fosgerau et al., 2013; Oyama and Hato, 2019; Oyama, 2023) with the definitions of $\mathbf{V}_{(t,w)}(\mathbf{p}) = [\exp\{-\phi\pi_{(k,i)}^{(t,w)}(\mathbf{p})\}]$ and $\mathbf{W}_t(\mathbf{p}) = [\exp\{-\phi(c_{ij}^D - p_{(t,j)})\}]$:

$$\mathbf{V}_{(k,i)}^{(t,w)}(\mathbf{p}) = \sum_{j \in \mathcal{J}} W_{t,i,j}(\mathbf{p}) \mathbf{V}_{(k+1,j)}^{(t,w)}(\mathbf{p}) \quad (4.16)$$

and the solution can be simply computed as shown in Section 5.2.

Moreover, the demand (network loading) functions with the MNL probabilities are

$$y_{j,t}^* = N_j^S \exp\{-\theta(C_{j,t}^S + p_{(t,j)} - \Pi_j^S(\mathbf{p}))\} \quad (4.17)$$

$$\begin{aligned} x_{k,i,j}^{(t,w)*} &= \left(\sum_{h \in \mathcal{J}} x_{k,hi}^{(t,w)*} \right) \cdot \exp\{-\phi(c_{ij}^D - p_{(t,j)} + \pi_{(k+1,j)}^{(t,w)}(\mathbf{p}) - \pi_{(k,i)}^{(t,w)}(\mathbf{p}))\}, \\ &= \left(\sum_{h \in \mathcal{J}} x_{k,hi}^{(t,w)*} \right) W_{t,i,j}(\mathbf{p}) \mathbf{V}_{(k+1,j)}^{(t,w)}(\mathbf{p}) / \mathbf{V}_{(k,i)}^{(t,w)}(\mathbf{p}) \end{aligned} \quad (4.18)$$

with which we can compute the task partition patterns given the price pattern $\mathbf{p}$.

4.3.2. Example 2: The nested logit model

We then show another example of the nested logit (NL) model that captures the correlation among unobserved utilities. Since both the models of shippers and drivers involve *out-out* options, we can consider the nest structure focusing on participation in CSD. That is, we assume that a shipper (driver) first chooses among two nests, i.e., whether or not to participate in the CSD system, and if participating, then the shipper (driver) chooses a time window for delivery (task bundle to perform). The participation nest for the shipper model includes all time window alternatives in $\mathcal{T}$, and that for the driver model consists of all task bundles in $\mathcal{R}$ for the driver model.

The NL formulations for the shipper model are:

$$\hat{H}_j^S(\mathbf{y}_j) = -\frac{1}{\theta} \left(y_{j,0} \ln \frac{y_{j,0}}{N_j^S} + \sum_{t \in \mathcal{T}} y_{j,t} \ln \frac{\sum_{t \in \mathcal{T}} y_{j,t}}{N_j^S} \right) - \frac{1}{\hat{\theta}} \sum_{t \in \mathcal{T}} y_{j,t} \ln \frac{y_{j,t}}{\sum_{t \in \mathcal{T}} y_{j,t}} \quad (4.19)$$

$$\Pi_j^S(\mathbf{p}) = -\frac{1}{\theta} \ln \left\{ \exp(-\theta C_{j,0}^S) + \exp(-\theta \Pi_{j,\mathcal{T}}^S(\mathbf{p})) \right\} \quad (4.20)$$

$$\Pi_{j,\mathcal{T}}^S(\mathbf{p}) = -\frac{1}{\hat{\theta}} \ln \sum_{t \in \mathcal{T}} \exp\{-\theta(C_{j,t}^S + p_{(t,j)})\} \quad (4.21)$$

where $\Pi_{j,\mathcal{T}}^S(\mathbf{p})$ represents the expected minimum cost given $\mathbf{p}$ among shipping time windows. Two scales θ and $\hat{\theta}$ are associated with nest choice and time window choice within the participation nest, respectively. As such, the demand functions are

$$y_{j,0}^* = N_j^S \exp\{-\theta(C_{j,0}^S - \Pi_j^S(\mathbf{p}))\} \quad (4.22)$$

$$y_{j,\mathcal{T}}^* = N_j^S \exp\{-\theta(\Pi_{j,\mathcal{T}}^S(\mathbf{p}) - S_j(\mathbf{p}))\} \quad (4.23)$$

$$y_{j,t}^* = y_{j,\mathcal{T}}^* \exp\{-\hat{\theta}(C_{j,t}^S + p_{(t,j)} - \Pi_{j,\mathcal{T}}^S(\mathbf{p}))\} \quad (4.24)$$

where $y_{j,\mathcal{T}}^*$ is the total number of shippers who outsource the delivery of task j , while $y_{j,0}^*$ is the number of shippers who do not participate in the CSD system.

The NL formulations for the driver model are obtained similarly. For clarity, we first show the path-based formulations as follows:

$$\hat{H}_{(t,w)}^D(\mathbf{f}_{(t,w)}) = -\frac{1}{\phi} \left(f_{\emptyset}^{(t,w)} \ln \frac{f_{\emptyset}^{(t,w)}}{N_{(t,w)}^D} + \sum_{r \in \mathcal{R}} f_{(t,w),r} \ln \frac{\sum_{l \in \mathcal{R}} f_{(t,w),l}}{N_{(t,w)}^D} \right) - \frac{1}{\hat{\phi}} \sum_{r \in \mathcal{R}} f_{(t,w),r} \ln \frac{f_{(t,w),r}}{\sum_{r \in \mathcal{R}} f_{(t,w),r}} \quad (4.25)$$

$$\Pi_{(t,w)}^D(\mathbf{p}) = -\frac{1}{\phi} \ln \left\{ \exp(-\phi C_{(t,w),\emptyset}^D) + \exp(-\phi \Pi_{(t,w),\mathcal{R}}^D(\mathbf{p})) \right\} \quad (4.26)$$

$$\Pi_{(t,w),\mathcal{R}}^D(\mathbf{p}) = -\frac{1}{\hat{\phi}} \ln \sum_{r \in \mathcal{R}} \exp \left\{ -\hat{\phi}(C_{(t,w),r}^D - p_{t,r}) \right\} \quad (4.27)$$

where $\Pi_{(t,w),\mathcal{R}}^D(\mathbf{p})$ represents the expected minimum cost given $\mathbf{p}$ among all possible task bundles. Like the shipper model, the NL driver model has two scales ϕ and $\hat{\phi}$ associated with nest and bundle choices, respectively. The demand functions are

$$f_{\emptyset}^{(t,w)\star} = N_{(t,w)}^D \exp \left\{ -\phi(C_{(t,w),\emptyset}^D - \Pi_{(t,w)}^D(\mathbf{p})) \right\} \quad (4.28)$$

$$f_{\mathcal{R}}^{(t,w)\star} = N_{(t,w)}^D \exp \left\{ -\phi(\Pi_{(t,w),\mathcal{R}}^D(\mathbf{p}) - \Pi_{(t,w)}^D(\mathbf{p})) \right\} \quad (4.29)$$

$$f_r^{(t,w)\star} = f_{\mathcal{R}}^{(t,w)\star} \exp \left\{ -\hat{\phi}(C_{(t,w),r}^D - p_{t,r} - \Pi_{(t,w),\mathcal{R}}^D(\mathbf{p})) \right\} \quad (4.30)$$

On the task-chain network, $f_{\emptyset}^{(t,w)} = x_{0,od}^{(t,w)}$, $f_{\mathcal{R}}^{(t,w)} = N_{(t,w)}^D - x_{0,od}^{(t,w)}$ and $C_{(t,w),\emptyset}^D = c_{od}$. Simply by removing edge (o, d) from the network, the MNL path choice model with scale $\hat{\phi}$ represents the within-nest choice model of the NL model, thereby

$$\pi_{(k,i)}^{(t,w)}(\mathbf{p}) = -\frac{1}{\hat{\phi}} \ln \sum_{j \in \mathcal{J}} \exp \left\{ -\hat{\phi}(c_{ij}^D - p_{(t,j)} + \pi_{(k+1,j)}^{(t,w)}(\mathbf{p})) \right\} \quad (4.31)$$

$$x_{k,ij}^{(t,w)\star} = \left(\sum_{h \in \mathcal{J}} x_{k,hi}^{(t,w)\star} \right) \cdot \exp \left\{ -\hat{\phi}(c_{ij}^D - p_{(t,j)} + \pi_{(k+1,j)}^{(t,w)}(\mathbf{p}) - \pi_{(k,i)}^{(t,w)}(\mathbf{p})) \right\}, \quad (4.32)$$

and $\Pi_{(t,w),\mathcal{R}}^D = \pi_{(0,o)}^{(t,w)}$ and $f_{\mathcal{R}}^{(t,w)\star} = \sum_{j \in \mathcal{J}} x_{0,oj}^{(t,w)\star}$ hold. Again, (4.31) and (4.32) reduce to

$$V_{(k,i)}^{(t,w)}(\mathbf{p}) = \sum_{j \in \mathcal{J}} W_{t,ij}(\mathbf{p}) V_{(k+1,j)}^{(t,w)}(\mathbf{p}) \quad (4.33)$$

$$x_{k,ij}^{(t,w)\star} = \left(\sum_{h \in \mathcal{J}} x_{k,hi}^{(t,w)\star} \right) \cdot W_{t,ij}(\mathbf{p}) V_{(k+1,j)}^{(t,w)}(\mathbf{p}) / V_{(k,i)}^{(t,w)}(\mathbf{p}), \quad (4.34)$$

with scale parameter $\hat{\phi}$. As such, the surplus and demand functions for the NL model can also be decomposed into edge-based formulations on the network.

5. Solution algorithm of the master problem

5.1. Accelerated gradient descent method

We exploit the dual formulation of [SO/Master-TAP-D] to develop an efficient solution algorithm. The primal problem [SO/Master-TAP] is a stochastic TAP but has capacity constraints. Solving a capacitated traffic assignment problem (CTAP) has been challenging, and solution algorithms applicable to large-scale networks have not been developed in the literature (Nie et al., 2004; Meng et al., 2008).

In contrast, the dual problem [SO/Master-TAP-D] is unconstrained with the price pattern $\mathbf{p}$ being the only decision variable, which allows us to apply existing fast non-linear optimization algorithms. This study presents an efficient dual algorithm based on the accelerated gradient descent (AGD) method (Nesterov, 1983; Beck and Teboulle, 2009). The AGD method is a first-order (gradient) method and requires less memory space compared to second-order methods. It also achieves the known complexity bound $\Omega(1/k^2)$, that is, in the worst case, any iterative method based solely on the function and gradient evaluations cannot achieve better accuracy than $\Omega(1/k^2)$ at iteration k . Although dual-type AGD methods are presented in the context of traffic equilibrium assignment (Oyama et al., 2022; Yao et al., 2024), this study is the first to propose to develop an AGD-based method for CTAP.

The gradient of G , the objective function (4.11) of [SO/Master-TAP-D], is obtained as follows:

$$\nabla_{p_{(t,j)}} G = y_{j,t}^* - \sum_{w \in \mathcal{W}} \sum_{k=0}^K \sum_{i \in \mathcal{J}} x_{k,ij}^{(t,w)\star} \quad (5.1)$$

This represents the gap between the number of permits for time window t to be issued for task j and the number of tasks for j to be partitioned for drivers traveling in time window t . Therefore, the gradients have an intuitive interpretation as they are equal to *excess demand* for the delivery of tasks in the time window.

We can obtain these flows $\mathbf{y}^*$ and $\mathbf{x}^*$ by performing network loading, and we write these as:

$$\mathbf{y}_j^* = F_j^S(\mathbf{p}) \quad (5.2)$$

$$\mathbf{x}_{(t,w)}^* = F_{(t,w)}^D(\mathbf{p}) \quad (5.3)$$

where F is a demand (network loading) function: see for examples (4.17) and (4.18) for the MNL model case and (4.22)–(4.24) and (4.32) for the NL model case.

Then, we apply the accelerated gradient projection (AGP) that integrates the proximal operator into the AGD method and updates a price/reward pattern by

$$\mathbf{p}^{(m+1)} = \text{Proj}_{\mathcal{P}}(\mathbf{p}^{(m)} + \gamma \nabla_{\mathbf{p}} G(\mathbf{p}^{(m)})) = \langle \mathbf{p}^{(m)} + \gamma \nabla_{\mathbf{p}} G(\mathbf{p}^{(m)}) \rangle_+ \quad (5.4)$$

where $\mathbf{b} = \langle \mathbf{a} \rangle_+$ denotes an element-wise operation $b_{(t,j)} = \max\{a_{(t,j)}, 0\}$. The solution algorithm for our problem [SO/Master-TAP-D] is summarized as follows.

[Accelerated Gradient Descent (AGD)]

Step 1: Initialization: Initialize price vector $\mathbf{p}^{(0)}$ and step size γ , and set iteration counter $m := 0$, adaptive restart counter $l := 0$, and auxiliary price $\hat{\mathbf{p}}^{(0)} := \mathbf{p}^{(0)}$.

Step 2: Network loading: Compute $\mathbf{y}_j^{(m)}$ for all $j \in \mathcal{J}$ and $\mathbf{x}_{(t,w)}^{(m)}$ for all $(t, w) \in \mathcal{T} \times \mathcal{W}$:

$$\begin{aligned} \mathbf{y}_j^{(m)} &= \mathcal{F}_j^S(\mathbf{p}^{(m)}) \\ \mathbf{x}_{(t,w)}^{(m)} &= \mathcal{F}_{(t,w)}^D(\mathbf{p}^{(m)}) \end{aligned}$$

Step 3: Updating: Update the current solution using a momentum:

$$\mathbf{p}_{(t,j)}^{(m+1)} = \langle \hat{\mathbf{p}}_{(t,j)}^{(m)} + \gamma \nabla_{\mathbf{p}_{(t,j)}} G(\mathbf{p}^{(m)}) \rangle_+ = \max \left\{ \hat{\mathbf{p}}_{(t,j)}^{(m)} + \gamma (y_{j,t}^{(m)} - \sum_{w \in \mathcal{W}} \sum_{k=0}^K \sum_{i \in \mathcal{J}} x_{k,i,j}^{(t,w)(m)}), 0 \right\} \quad (5.5)$$

$$\tau_{l+1} = \frac{1 + \sqrt{1 + 4\tau_l^2}}{2} \quad (5.6)$$

$$\hat{\mathbf{p}}^{(m+1)} = \mathbf{p}^{(m+1)} + \frac{\tau_l - 1}{\tau_{l+1}} (\mathbf{p}^{(m+1)} - \mathbf{p}^{(m)}) \quad (5.7)$$

Step 4: Adaptive restart: If the restart criterion holds, then $l := 0$. Otherwise, $l := l + 1$.

Step 5: Convergence test: If the convergence criterion holds, stop the algorithm. Otherwise, set $m := m + 1$ and return to Step 1.

The efficiency of the AGD method relies on network loading (i.e., the calculation of gradients) in Step 2. Therefore, the next subsection explains the efficient network loading algorithm.

5.2. Network loading algorithm

The computation of the shipper flow pattern $\mathbf{y}$ requires only a single-stage calculation of shippers' time-window choice probabilities $\mathbf{P}^S$ given a price pattern $\mathbf{p}$. Then, multiplying this probability $P_j^S(t)$ by the number of shippers N_j^S yields $y_{j,t}$ (see (4.17) and (4.22)–(4.24) for the MNL and NL models, respectively). The driver model is not as simple as the shipper model, because the choice set (i.e., the set of task bundles) for drivers cannot be easily enumerated. Therefore, we apply the Markovian traffic assignment (MTA) algorithm (Dial, 1971; Bell, 1995; Akamatsu, 1996; Oyama and Hato, 2019; Oyama et al., 2022) that efficiently computes network traffic flows without enumerating paths. The MTA algorithm consists of two main phases: (i) the backward computation of the surplus (expected maximum utility) functions and (ii) the forward computation of edge flows, as described as follows (we focus on a specific driver group (t, w) and omit the subscript for simplicity).

[Markovian traffic assignment (MTA)]

Step 1: Initialization: Input a reward pattern $\mathbf{p}$ and the number of drivers $N_{(t,w)}$, and pre-compute the matrix of generalized costs

$\mathbf{W} = [\exp\{-\phi(c_{ij}^D - p_{(t,j)})\}]$. Initialize the exponential of surplus functions $\mathbf{V} = [\exp(-\phi\pi_{(k,j)})] = \mathbf{0}$ and edge flows $\mathbf{x} = [x_{k,i,j}] = \mathbf{0}$ to all zero.

Step 2: Backward computation:

Step 2-1: Update $V_{(k,d)} = 1$ for all $k \in \{0, 1, \dots, K+1\}$, and then set $k = K$.

Step 2-2: For all $j \in \mathcal{J}$, compute:

$$V_{(k,j)} = \sum_{j' \in \tilde{\mathcal{J}}} W_{jj'} V_{(k+1,j')} \quad (5.8)$$

Step 2-3: If $k = 0$, go to Step 3. Otherwise, update $k := k - 1$ and go back to Step 2-2.

Step 3: Forward computation:

Step 3-1: Set $k = 1$, and compute for all $j \in \tilde{\mathcal{J}}$:

$$x_{0,o,j} = N_{(t,w)} W_{oj} V_{(1,j)} / V_{(0,o)}. \quad (5.9)$$

Step 3-2: For all $ij \in \mathcal{E}$, compute:

$$x_{k,ij} = \left(\sum_{h \in \mathcal{J}} x_{k-1,hi} \right) W_{ij} V_{(k+1,j)} / V_{(k,i)} \quad (5.10)$$

Step 3-3: If $k = K$, finish the algorithm. Otherwise, update $k := k + 1$ and go back to **Step 3-2**.

Note that although the algorithm presented above is for the MNL model, the NL model requires only an additional step in **Step 3-1** to first partition the flow $N_{(t,w)}$ into participating/non-participating drivers. This MTA algorithm on the task-chain network is very efficient because both forward and backward computation can be performed by vector multiplication and only requires K times computation. This algorithm is a classical method for link-based stochastic network loading in transportation science, but the novelty of this paper lies in effectively utilizing this algorithm in solving the CSD matching problem.

6. Solution algorithm of the sub-problems

To solve our sub-problems, we exploit the VCG market mechanism (Vickrey, 1961; Clarke, 1971; Groves, 1973), known as a truthful and efficient mechanism (Cramton et al., 2006; Vohra, 2011). This section shows that we can design the VCG mechanisms for both the shipper and driver markets and that they satisfy the two desirable properties of truth-telling and efficiency. The truth-telling property ensures that the platform manager directly observes the private utilities of shippers and drivers through their bids. That is, by repeating the auctions on a daily basis the manager can collect sufficient information to estimate the utility distributions.

6.1. Shipper market

First, we design the VCG mechanism for shipper markets as follows.

[VCG/Shipper]

Step 1: Each shipper $b \in B_j$ submits a bid s_t^b for each time window t , indicating their willingness to pay for the delivery in the window.

Step 2: The platform assigns a shipping permit to each driver to maximize the social surplus with respect to the declared values, by solving [SO/Sub-Shipper(j)] with c_t^b in (3.12) replaced with $-s_t^b$:

$$\begin{aligned} \max_{\mathbf{n}_{(j)}} \quad & z_j^S(\mathbf{n}_{(j)} | \mathbf{s}, \mathbf{y}) \equiv \sum_{b \in B_j} \sum_{t \in \tilde{\mathcal{T}}} n_t^b s_t^b \\ \text{subject to} \quad & \sum_{t \in \tilde{\mathcal{T}}} n_t^b = 1 \quad \forall b \in B_j \quad (3.13) \\ & \sum_{b \in B_j} n_t^b = y_{j,t} \quad \forall t \in \tilde{\mathcal{T}} \quad (3.14) \\ & n_t^b \in \{0, 1\} \quad \forall b \in B_j, \forall t \in \tilde{\mathcal{T}} \quad (3.15) \end{aligned} \quad (6.1)$$

Step 3: The price p_b that shipper $b \in B_j$ pays is determined by

$$p_b(\mathbf{s}_b, \mathbf{s}_{-b}) = \max_{\mathbf{n}_{-b}} z_{j,-b}^S(\mathbf{n}_{-b} | \mathbf{s}_{-b}, \mathbf{y}) - \sum_{b' \in B_j \setminus \{b\}} \sum_{t \in \tilde{\mathcal{T}}} n_t^{b' \star}(\mathbf{s}) s_t^{b'} \quad (6.2)$$

where $\mathbf{n}^*(\mathbf{s}) = [n_t^{b \star}(\mathbf{s})]_{t \in \tilde{\mathcal{T}}, b \in B_j}$ is the optimal matching pattern under bid $\mathbf{s} = (\mathbf{s}_b, \mathbf{s}_{-b})$ for the problem in Step 2, and the subscript “ $-b$ ” represents all shippers in B_j except for b . The first and second terms of the RHS of (6.2) are the surplus of shippers $-b$ achieved without and with the participation of b , respectively.

Herein, Eq. (6.2) implies that each shipper will pay based on the loss in social surplus caused by the shipper’s participation, i.e., the compensation to the opportunity loss of someone else. Under this payment, the VCG mechanism holds efficiency properties.

Proposition 6.4. *The VCG mechanism for the shipper markets [VCG/Shipper] is truthful and efficient.*

Proof. Appendix D. $\square$

This property allows us to collect information on privately perceived costs of shippers by performing the auctions in the sub-problems.

6.2. Driver market

Using the task-chain network representation, we reformulate the sub-problem [SO/Sub-Driver(t, w)]:

[SO/Sub-Driver-Edge(t, w)]

$$\min_{\mathbf{m}_{(t,w)}} z^D(\mathbf{m}_{(t,w)} | \mathbf{x}_{(t,w)}) \equiv \sum_{a \in \mathcal{A}_{(t,w)}} \sum_{k=0}^K \sum_{ij \in \mathcal{E}} m_{k,ij}^a c_{ij}^D \quad (6.3)$$

$$\text{subject to } \sum_{i \in \mathcal{J}} m_{k-1,ij}^a - \sum_{l \in \mathcal{J}} m_{k,jl}^a = \eta_{k,j} \quad \forall k \in \{0, \dots, K+1\}, \forall j \in \mathcal{J}, \forall a \in \mathcal{A}_{(t,w)} \quad (6.4)$$

$$\sum_{a \in \mathcal{A}_{(t,w)}} m_{k,ij}^a = x_{k,ij}^{(t,w)} \quad \forall k \in \{0, \dots, K+1\}, \forall ij \in \mathcal{E} \quad (6.5)$$

$$m_{k,ij}^a \in \{0, 1\} \quad \forall k \in \{0, \dots, K+1\}, \forall ij \in \mathcal{E}, \forall a \in \mathcal{A}_{(t,w)} \quad (6.6)$$

where $\mathbf{m}_{(t,w)} = [m_{k,ij}^a]$ is a binary variable that takes 1 when driver $a \in \mathcal{A}_{(t,w)}$ is matched with edge ij for k th choice and 0 otherwise, and $\mathbf{x}_{(t,w)}$ is the task partition pattern that is determined in the master TAP problem.

The platform opens an auction that distributes delivery tasks among each driver group $\mathcal{A}_{(t,w)}$. In this study, a driver may perform a bundle of delivery tasks, but the number of possible bundles is generally large. Therefore, we utilize the above edge-based formulation to design a scalable VCG auction mechanism.

[VCG/Driver]

Step 1: Each CSD driver $a \in \mathcal{A}_{(t,w)}$ submits a bid $\tilde{c}_{ij}^a$ for task pair $(i, j) \in \mathcal{E}$, indicating their willingness to accept for performing the task j given their location i^- .

Step 2: The platform matches delivery tasks to each driver to minimize the social cost with respect to the declared costs $\tilde{\mathbf{c}}$, by solving [SO/Sub-Edge(t, w)] with c_{ij}^a in (6.3) replaced with $\tilde{c}_{ij}^a$:

$$\min_{\mathbf{m}_{(t,w)}} z^D(\mathbf{m}_{(t,w)} | \tilde{\mathbf{c}}, \mathbf{x}) \equiv \sum_{a \in \mathcal{A}_{(t,w)}} \sum_{k=0}^K \sum_{ij \in \mathcal{E}} m_{k,ij}^a \tilde{c}_{ij}^a \quad (6.7)$$

$$\text{subject to } (6.4), (6.5), (6.6). \quad (6.8)$$

Step 3: The reward w_a that CSD driver $a \in \mathcal{A}_{(t,w)}$ receives is determined by

$$p_a(\tilde{\mathbf{c}}_a, \tilde{\mathbf{c}}_{-a}) = \min_{\mathbf{m}_{-a}} z_{-a}^D(\mathbf{m}_{-a} | \tilde{\mathbf{c}}_{-a}, \mathbf{x}) - \sum_{a' \in \mathcal{A}_{(t,w)} \setminus \{a\}} \sum_{k=0}^K \sum_{ij \in \mathcal{E}} m_{k,ij}^{a'*}(\tilde{\mathbf{c}}) \tilde{c}_{ij}^{a'} \quad (6.9)$$

where $\mathbf{m}^*(\tilde{\mathbf{c}}) = [m_{k,ij}^{a'*}(\tilde{\mathbf{c}})]_{i,j \in \mathcal{J}, k \in \mathcal{K}, a \in \mathcal{A}_{(t,w)}}$ is the optimal matching pattern under bid pattern $\tilde{\mathbf{c}}$. The subscript “ $-a$ ” denotes all drivers except for a , and thus $\mathbf{b}_{-a}$ represents the vector of bids excluding a , and $\min_{\mathbf{m}_{-a}} z_{-a}^D(\cdot)$ is the social cost value achieved without participation of a .

Eq. (6.9) implies that each driver is rewarded based on the driver’s contribution to the reduction of social cost, and the reward would be zero if the driver did not contribute to the system. The edge-based VCG mechanism for drivers’ markets also holds efficiency properties.

Proposition 6.5. *The VCG mechanism for the driver markets [VCG/Driver] is truthful and efficient.*

Proof. Appendix E. $\square$

Again, this property allows us to collect information on privately perceived costs of drivers by performing auctions in the market.

7. Reduction of computational complexity

Our approach drastically reduces the computational complexity of the matching problem. Before presenting numerical results, we discuss how much our approach theoretically reduces the complexity and required memory capacity compared to the original matching problem. We refer to Fig. 2 to see how our approach transformed the problem and reduced its size.

The original matching problem [SO] simultaneously decides matchings between drivers and task bundles and between shippers and delivery time windows. The decision variables are $\mathbf{m}$ and $\mathbf{n}$ with sizes of $|\mathcal{A}| \times |\tilde{\mathcal{R}}|$ and $|\mathcal{B}| \times |\tilde{\mathcal{T}}|$, which can be tremendously large in urban areas due to the growth of $|\mathcal{A}|$ and $|\mathcal{B}|$. The number of task bundles $|\tilde{\mathcal{R}}|$ exponentially increases as $|\mathcal{J}|$ and K increase.

To make this problem tractable, we first apply the FPD approach that decomposes [SO] into the fluidly approximated master problem [SO/Master-A] and small-scale sub-problems [SO/Sub-Shipper(j)] and [SO/Sub-Driver(t, w)], by grouping individual shippers by task pickup-delivery location pairs ($\mathcal{B} \rightarrow \{\mathcal{B}_j\}_{j \in \mathcal{J}}$) and individual drivers by OD pairs and time windows ($\mathcal{A} \rightarrow \{\mathcal{A}_{(t,w)}\}_{(t,w) \in \mathcal{T} \times \mathcal{W}}$). The decision variables of the master problem are then $\mathbf{y}$ and $\mathbf{f}$ with sizes of $|\mathcal{J}| \times |\tilde{\mathcal{T}}|$ and $|\mathcal{T}| \times |\mathcal{W}| \times |\tilde{\mathcal{R}}|$.

Then, we transform the master problem into a TAP on the task-chain network [SO/Master-TAP]. This transformation exploits Markovian traffic assignment formulation to address the computational complexity of task-bundling. That is, we do not need to

enumerate $\tilde{\mathcal{R}}$ but implicitly calculate $\mathbf{f}$ based on edge flows $\mathbf{x}$. Furthermore, we propose solving its dual problem [SO/Master-TAP-D] whose decision variable is just $\mathbf{p}$ with a size of $|\mathcal{T}| \times |J|$. The AGD method finds $\mathbf{p}$ that maximizes the (fluidly approximated) social surplus while satisfying the demand-supply constraint.

The sub-problems are much smaller problems than the original problem [SO]. In addition, all the sub-problems are completely independent of each other and can be solved in parallel. The driver-side sub-problem [SO/Sub-Driver(t, w)] is transformed using the task-chain network representation into [SO/Sub-Edge(t, w)] and can be solved without bundle generation. It is worth noting that the network size for [SO/Sub-Edge(t, w)] depends on the result of the task partition of the master problem. If only a few categories of tasks are assigned to a driver group $|\mathcal{A}_{(t,w)}|$, only a small sub-network of Fig. 3 needs to be considered and thus the decision variable of [SO/Sub-Edge(t, w)] becomes very small.

To summarize, our approach transformed the original intractable large-scale problem [SO] into [SO/Master-TAP-D], only finding $\mathbf{p}$ by an efficient AGD method, and independent small-scale sub-problems [SO/Sub-Shipper(j)] and [SO/Sub-Edge(t, w)]]. By doing so, we drastically reduce computational complexity and save memory space.

Importantly, our approach does not only scale down the size of the problem. The master problem evaluates the optimal value functions of the sub-problems as its objective, by exploiting the RUM framework, so that global matching efficiency is achieved. As such, our approach enables obtaining an accurate solution to the large-scale CSD matching problem with high speed.

8. Numerical experiments

This section presents numerical experiments to demonstrate the performance of the proposed mechanism and algorithm, in terms of both computational efficiency and approximation accuracy.

We consider the original solution of [SO] as the ground-truth benchmark, where the perceived costs c_r^a and c_i^b of all drivers and shippers are perfectly known. That is, this solution achieves globally optimal matching. We refer to this mechanism as a *naive mechanism with perfect information* (N1).

In addition, to discuss the impact of the observations of drivers' and shippers' private utilities through auctions, we consider another benchmark mechanism. Here, we assume that information on drivers' utilities is unavailable, and the platform solves matching by solving [SO] using only the deterministic cost $C_{(t,w),r}^D$ and $C_{j,t}^S$. We call this mechanism a *naive mechanism without private information* (N0).

Both N1 and N0 mechanisms are solved as a linear programming (LP) model. We use an LP solver of Gurobi Optimizer version 10.0.1 with the default parameter settings and record CPU time in seconds.

Then, we compare the proposed FPD mechanism with the benchmarks. The FPD mechanism solves shipper-driver matching by utilizing the distributions of shippers' and drivers' private utilities, which are assumed to have been estimated using information collected during past auctions. As described in Section 6, the truth-telling property of the auction mechanisms ensures the collection of reliable information.

The master task-partition problem is solved by the AGD method described in Section 5, and the sub-problems are solved by the same LP solver as the benchmarks. We use the MNL model for this experiment, and terminate the AGD method when the iteration reaches 1000 times or when the following three criteria are all satisfied: the maximum gradient G is below 0.1; the maximum relative difference between $\mathbf{p}^{(m)}$ and $\mathbf{p}^{(m+1)}$ is below 10^{-4} ; and the relative objective difference is below 10^{-6} . We set the step size γ of the AGD method to 0.001 after some initial trials.

We perform the experiments in a Python 3.9 environment on a machine with a 12-core Apple M2 Pro and 32 GB of memory, and implement the algorithms by writing our own code.

8.1. Performance indicators

We evaluate the performance of the FPD mechanism by using the following indicators:

- **CPU time.** The CPU time of the FPD mechanism is the sum of the runtime of the AGD method to solve [SO/Master-TAP-D], the average computing time taken to solve [SO/Sub-Shipper(j)] by the LP solver, and that for [SO/Sub-Driver(t, w)]]. The reasoning behind taking the average over the sub-problems is that the sub-problems are completely independent of each other and can be solved in parallel. The CPU times of the N1 and N0 mechanisms are the runtimes to solve [SO] by the LP solver. Note that since enumerating task bundles is computationally prohibitive, we solve [SO] and [SO/Sub-Driver(t, w)] using the edge-based formulation on the task-chain network (see Appendix F).²
- **Social surplus.** The optimal objective value z_1^* of [SO] achieved by the N1 mechanism is the ground-truth social surplus that we aim to accurately approximate. The N0 mechanism also solves [SO] but without private information. We define the objective achieved by N0 as $\bar{z}_0^*$, and measure its error compared to the ground truth by the true relative difference: $(z_1^* - \bar{z}_0^*)/z_1^*$. The social surplus achieved by the FPD mechanism is the sum of the optimal objective values of all sub-problems, i.e., $z_{\text{FPD}}^* = \sum_{j \in J} z_j^{S*} + \sum_{t \in \mathcal{T}} \sum_{w \in \mathcal{W}} z_{(t,w)}^{D*}$. We measure the approximation error by $(z_1^* - z_{\text{FPD}}^*)/z_1^*$.

² This means that the naive mechanisms also benefit from the edge-based formulation on the task-chain network. We show that even with this reformulation of [SO], the naive mechanisms still require large computational costs, and our approach drastically reduces the costs.

- **Price/reward patterns.** We additionally compare the price/reward patterns between mechanisms. The optimal reward pattern of the N1 mechanism $\mathbf{p}_1^*$ is obtained as the Lagrangian multipliers by solving [SO]. We obtain $\mathbf{p}_1^*$ for the N0 mechanism in the same way but without private information. The reward pattern of the FPD mechanism $\mathbf{p}_{\text{FPD}}^*$ is obtained directly by solving the master problem as described in Section 5. We measure the approximation error of the reward pattern by the mean of relative differences $\mathbb{E}[(\mathbf{p}_1^* - \mathbf{p}_{\text{FPD}}^*)/\mathbf{p}_1^*]$ and $\mathbb{E}[(\mathbf{p}_1^* - \mathbf{p}_0^*)/\mathbf{p}_1^*]$.

8.2. Data generation procedure

The experiments use Winnipeg network data of [Transportation Networks for Research Core Team \(2023\)](#), which contains 1,052 nodes, 2,836 links, 147 zones, and 4,345 possible zone pairs for drivers' OD pairs or tasks' pickup-delivery locations. Based on this network, we generate datasets for the experiments by the following procedure:

1. Randomly generate $|\mathcal{W}|$ different driver OD pairs and $|\mathcal{J}|$ different task OD pairs from the Winnipeg network zone pairs.
2. Randomly decide the OD pair and time window of each of all $|\mathcal{A}|$ drivers and the pickup-delivery location pair for each of all $|\mathcal{B}|$ shippers, so that there is at least one driver for every $(t, w) \in \mathcal{T} \times \mathcal{W}$ and one shipper for every $j \in \mathcal{J}$.
3. Compute the true perceived costs: $c_t^b = C_{j,t}^S - \zeta_t^b$ for all shippers $b \in \mathcal{B}$ and time windows $t \in \mathcal{T}$; $c_{ij}^a = c_{ij}^D + \epsilon_{ij}^a$ for all drivers $a \in \mathcal{A}_{(t,w)}$ and edges $ij \in \mathcal{E}$. As this experiment assumes the MNL model, we randomly draw them from i.i.d. type I EV distributions $\zeta_t^b \sim \text{EV}(0, \theta)$, and $\epsilon_{ij}^a \sim \text{EV}(0, \phi)$.

We generate 20 datasets for each set of parameters tested in the experiments. The true private utility values generated in *Step 3* are directly used in the N1 mechanism to obtain the socially optimal solution. We assume that the FPD mechanism only knows the distributional parameters θ and ϕ , and the N0 mechanism does not have any information on private utility, i.e., $c_t^b = \epsilon_{ij}^a = 0$. This assumption of N0 is common with the previous matching models that consider deterministic compensation and neglect the stochastic behavior of shippers/drivers (e.g., [Wang et al., 2016](#); [Archetti et al., 2016](#); [Wang et al., 2023](#)).

8.3. Results

Defining the default setting as $(|\mathcal{A}|, |\mathcal{B}|, |\mathcal{T}|, |\mathcal{W}|, |\mathcal{J}|, K, \theta, \phi) = (5000, 5000, 4, 10, 10, 2, 1.0, 1.0)$, we see how the CPU time and approximation errors change depending on parameter values. We mainly focus on the effects of (1) the numbers of drivers and shippers $(|\mathcal{A}|, |\mathcal{B}|)$, (2) the number of task pickup-delivery location pairs $(|\mathcal{J}|)$, and (3) scale parameter values (θ, ϕ) .

CPU time. [Fig. 5](#) shows the CPU times required for N1 and the FPD mechanisms, with averages and standard deviations. First, the increase in the numbers of drivers $|\mathcal{A}|$ and shippers $|\mathcal{B}|$ directly enlarges the problem size and requires more CPU time for the N1 mechanism. In contrast, the CPU time of the FPD mechanism is reduced with $|\mathcal{A}|$ and $|\mathcal{B}|$. Because our master problem is cast as a flow-independent TAP (as in [Section 4.2](#)), its computational efficiency does not depend on $|\mathcal{A}|$ and $|\mathcal{B}|$. Then the increase in $|\mathcal{A}|$ and $|\mathcal{B}|$ tends to lead to a faster convergence of the AGD method. The sizes of the sub-problems increase, but their effects are not large enough to impact the total CPU time. Even at $|\mathcal{A}| = |\mathcal{B}| = 100000$, the FPD mechanism requires only 7.3 s on average, which is about 700 times faster than the N1 mechanism with 5008.7 s required on average. This result clarifies a substantial reduction of required computational effort by the proposed approach.

Second, as shown in panel (b) of [Fig. 5](#), the effect of the scale parameter (θ, ϕ) on CPU times is not large. The convergence of the master problem is faster with smaller scales due to smoothing, while the sub-problems are more efficiently solved with larger parameter values.

Third, the number of task pickup-delivery location pairs $|\mathcal{J}|$ and the maximum number K of tasks for each driver have a large impact on computational efficiency, because these parameters define the size of the task-chain network (i.e., the number of possible task bundles). Therefore, we test the effect of $\mathcal{J}$ with different values of K , as shown in panels (c-1) to (c-3). As a result, the CPU time of the N1 mechanism exponentially increases with the increase in $|\mathcal{J}|$. When $K = 2$ and 3, the average CPU time of N1 exceeds 1,000 s with large numbers of $|\mathcal{J}|$ (despite using the edge-based formulations). Although the CPU time of the FPD mechanism also increases, its increase is approximately linear in $|\mathcal{J}|$ and much more modest than that of the N1 mechanism. For instance, our mechanism requires only 16.2 s on average when $|\mathcal{J}| = 40$ and $K = 3$, which is 330 times more efficient than the N1 mechanism with 5,357.0 s on average. Note that the CPU time of the shippers' sub-problems decreases as $|\mathcal{J}|$ increases because the number of shippers per shipper group decreases.

Approximation errors. [Fig. 6](#) shows the approximation errors of the FPD mechanism, with the N1 mechanism considered the ground-truth reference. Overall, the FPD mechanism approximates the N1 mechanism with high accuracy. In most cases, the approximation error of the objective is less than 0.5%, and that of the price/reward value is below 2%.

We then discuss the effects of different parameters. Firstly, panel (a) shows the changes in the approximation errors according to the numbers of drivers and shippers (assumed $|\mathcal{A}| = |\mathcal{B}|$). The objective error decreases as an increase in $|\mathcal{A}|$ and $|\mathcal{B}|$ when these numbers are small. This is because the continuous distributions better approximate the true perceived costs with larger numbers of drivers and shippers. The error becomes 0.3% on average at $|\mathcal{A}| = |\mathcal{B}| = 2000$, and it continues slightly decreasing as $|\mathcal{A}|$ and $|\mathcal{B}|$ get larger. Yet, the FPD mechanism achieves the objective error of 2.3% even when $|\mathcal{A}| = |\mathcal{B}| = 200$, the smallest value. The price/reward error does not change with the sizes of $|\mathcal{A}|$ and $|\mathcal{B}|$, but the error is always below 2%. Secondly, panel (b) shows the changes in the approximation errors with the number of task pickup-delivery location pairs $|\mathcal{J}|$. Because the number of shippers

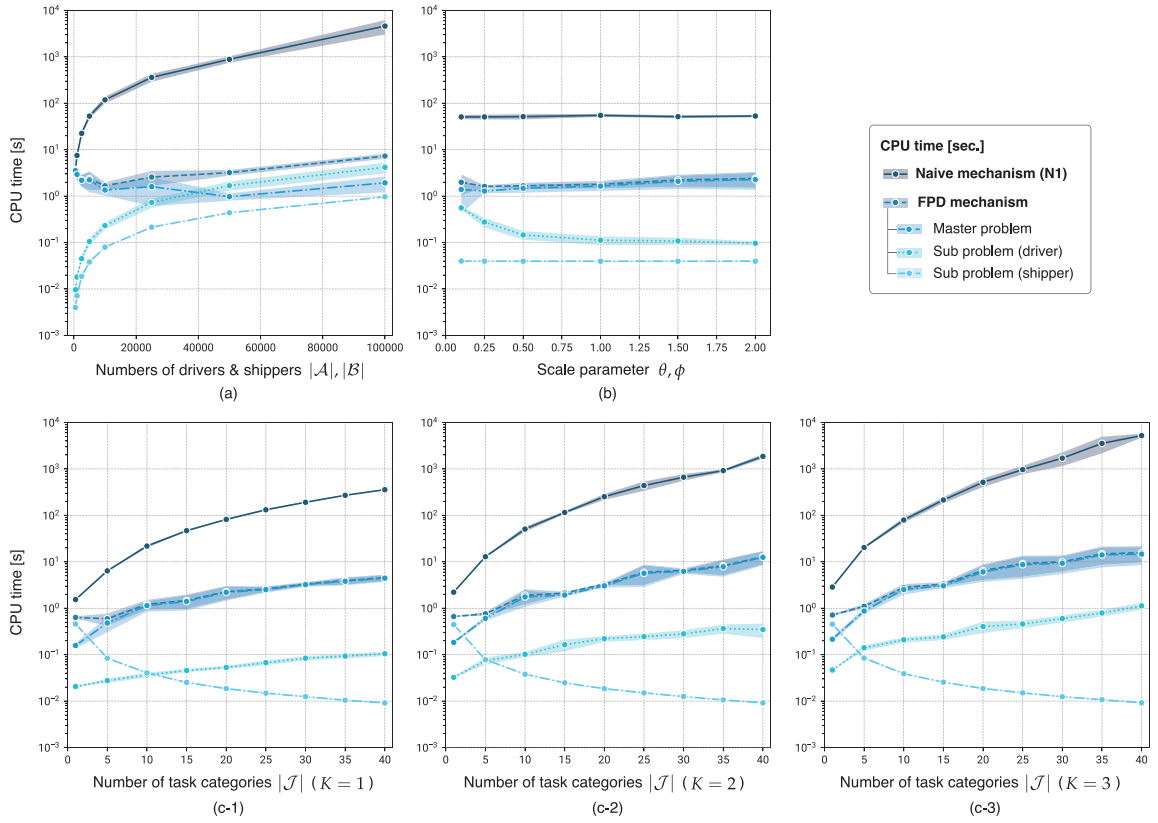


Fig. 5. CPU time.

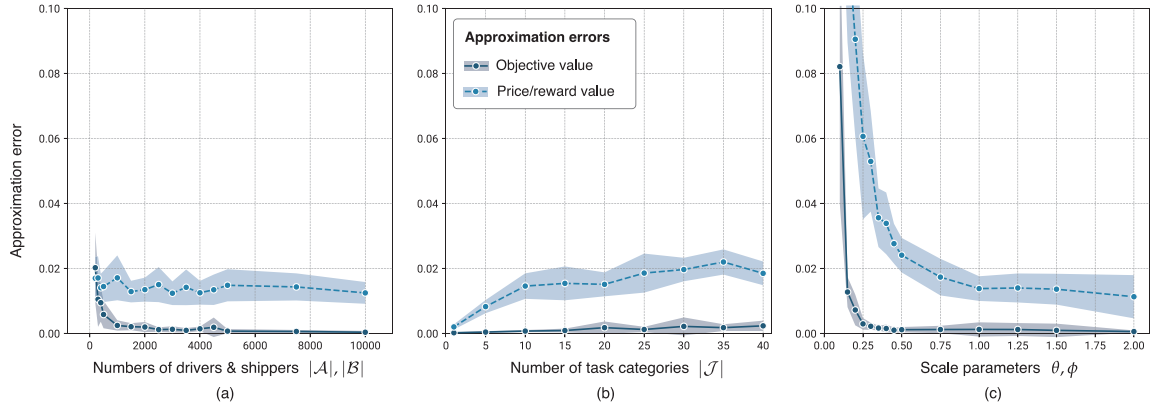


Fig. 6. Approximation errors.

per OD pair decreases as $|\mathcal{J}|$ increases, the continuous approximation gets less accurate; however, the change in the errors is very slight, and the objective error is 0.25% even when $|\mathcal{J}| = 40$.

Finally, panel (c) analyzes the effect of scale parameters θ and ϕ on approximation errors. As larger values of θ and ϕ indicate less heterogeneity in shippers' and drivers' preferences, the FPD mechanism better approximates the socially optimal matching pattern when the scales are larger than certain values. When the scales are small, the approximation errors become large, indicating the need to collect more private cost information through auctions. Still, the objective error is below 2% when $\theta = \phi = 0.15$ or larger.

Overall, these results demonstrate the high accuracy of the proposed approach for approximating the socially optimal matching patterns.

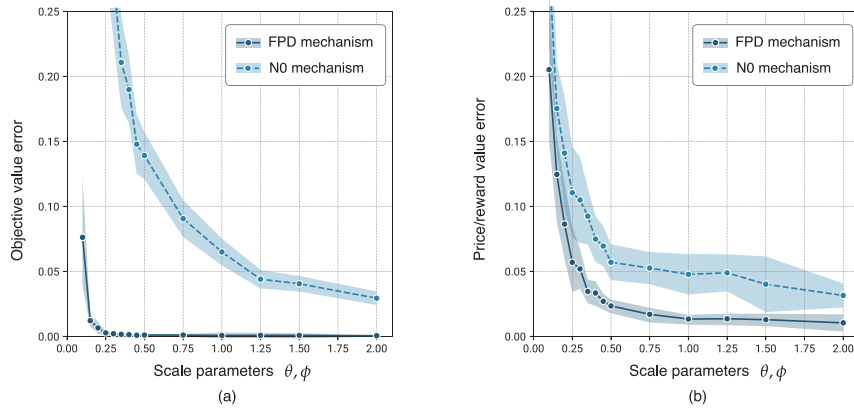


Fig. 7. Approximation error comparison between FPD and NO mechanisms.

Comparison with the NO mechanism. We finally compare the proposed FPD mechanism with NO mechanism in terms of approximation errors, to see the impact of capturing heterogeneity. Fig. 7 shows the results: (a) objective errors and (b) price/reward errors, where those of the FPD mechanism are the same as in Fig. 6(c). The approximation errors of NO mechanism highly depend on the level of heterogeneity in the preferences of drivers and shippers. The NO mechanism achieves errors less than 5% when $\theta = \phi = 1.25$ or larger, i.e., the behavior of shippers and drivers is relatively deterministic. However, the error becomes large with a high level of heterogeneity, and both objective and price/reward errors are above 25% when $\theta = \phi = 0.1$. In contrast, the FPD approach achieves highly accurate solutions even with a high level of heterogeneity, by utilizing the continuous distributions of perceived costs of individual drivers and shippers.

9. Concluding remarks

This paper presents a general formulation of a two-sided CSD matching problem and proposes a set of methodologies to solve the problem. We first reveal that the FPD approach of Akamatsu and Oyama (2024) can be extended to our general formulation. This approach not only addresses computational complexity but also allows us to collect information on privately perceived costs for each shipper/driver through VCG auctions. We further transform the master (task partition) problem into a TAP based on a task-chain network representation, which overcomes the difficulty of enumerating task bundles. Then, we formulate the dual problem of the TAP. This enables us to develop an efficient AGD method that only requires a price/reward pattern at market equilibrium, which drastically reduces memory space and computational cost. The numerical experiments demonstrate that our approach solves the general and complex CSD matching problem with high speed (~ 700 times faster) and accuracy (mostly within 0.5% errors).

It should be noted that this study assumes the platform manager's access to the utility distributions of shippers and drivers. We base this assumption on the fact that our auction mechanisms satisfy the truth-telling property and allow for the observation of true utilities through bids. To validate its practicality and reliability, our future work will conduct experiments to observe private utilities and estimate their distributions through auctions.

Finally, we remark that our approach is tested with an MNL model but is compatible with other random/perturbed utility models, including the network multivariate extreme value model (Oyama et al., 2022; Akamatsu et al., 2023). In addition, since CSD and ridesharing systems share a certain similarity (Furuhata et al., 2013; Alnaggar et al., 2021), the proposed framework can also be useful in the ridesharing context where ordinary drivers serve multiple passengers during their trips through a platform. In particular, our flow-based matching (task-partition) algorithm with agents' preference heterogeneity can fit the stochastic ridesharing equilibrium assignment model with OD-based ride-matching by Du et al. (2024). Such an extension/application of our approach to other related contexts is an important next step of this study.

CRedit authorship contribution statement

Yuki Oyama: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis, Data curation, Funding acquisition. **Takashi Akamatsu:** Writing – review & editing, Validation, Supervision, Methodology, Funding acquisition, Formal analysis, Conceptualization, Writing – original draft.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Table 1
Notation in this paper.

Category	Symbol	Description
Indices	a	Driver
	b	Shipper
	j, i	Task category (pickup-delivery location pair)
	w	Origin-destination (OD) pair for driver
	t	Time-window
	r	Bundle of tasks
	k	Order of a task being executed
Sets	$\mathcal{A}$	Set of drivers
	$\mathcal{A}_{(t,w)}$	Set of drivers who travel between OD pair w in time-window t
	$\mathcal{B}$	Set of shippers
	$\mathcal{B}_j$	Set of shippers who want to ship parcel between pickup-delivery location pair j
	$\mathcal{W}$	Set of OD pairs of drivers
	$\mathcal{J}$	Set of pickup-delivery location pairs of delivery tasks
	$\mathcal{T} (\tilde{\mathcal{T}})$	Set of time windows (with opt-out 0)
	$\mathcal{R} (\tilde{\mathcal{R}})$	Set of task bundles (with opt-out $\emptyset$)
Parameters	$N_{(t,w)}^D$	Number of drivers in $\mathcal{A}_{(t,w)}$
	N_j^S	Number of shippers in $\mathcal{B}_j$
	K	Maximum number of tasks each driver can perform
	θ	Scale of type I EV distribution for shippers' perceived costs
	ϕ	Scale of type I EV distribution for drivers' perceived costs
Functions	z	Objective function of [SO]
	z_j^S	Objective function of shipper sub problem [SO/Sub-Shipper(j)]
	$z_{(t,w)}^D$	Objective function of driver sub problem [SO/Sub-Driver(t, w)]
	Π_j^S	Surplus function of shippers in $\mathcal{B}_j$
	$\Pi_{(t,w)}^D, \pi_{(t,w)}^D$	Surplus function of drivers in $\mathcal{A}_{(t,w)}$
	$\mathcal{H}_j^S, \hat{\mathcal{H}}_j^S$	Perturbation function of shippers in $\mathcal{B}_j$
	$\mathcal{H}_{(t,w)}^D, \hat{\mathcal{H}}_{(t,w)}^D$	Perturbation function of drivers in $\mathcal{A}_{(t,w)}$
Variables	n_r^b	0–1 variable that takes 1 when shipper b chooses time window t and 0 otherwise
	m_r^a	0–1 variable that takes 1 when driver a is matched with task bundle r and 0 otherwise
	$m_{k,i,j}^a$	0–1 variable that takes 1 when driver a is matched with edge ij for k th choice and 0 otherwise
	$y_{j,t}$	Number of shipping permits of delivery time window t issued for shipper group $\mathcal{B}_j$
	$f_{(t,w),r}$	Number of task bundles of r allocated to driver group $\mathcal{A}_{(t,w)}$
	$x_{k,i,j}^{(t,w)}$	Number of drivers in $\mathcal{A}_{(t,w)}$ to be matched with edge ij for k th choice
	$p_{(t,j)}$	Price/reward for shipping task j in time window t
	c_t^b	Perceived cost for shipper b shipping task in time window t
	$C_{j,t}^S$	Deterministic cost for shippers shipping task j in time window t
	c_r^a	Perceived cost for driver a taking detour to perform task bundle r
	$C_{(t,w),r}^D$	Deterministic detour cost for drivers who travel between w in t performing task bundle r
	$\delta_{r,j}$	Number of tasks between pickup-delivery location pair j included in task bundle r
	$\delta_{k,i,j}^r$	0–1 variable that takes 1 if path r uses edge (i, j) at k on the task-chain network and 0 otherwise.
	$\eta_{k,j}$	Task-chain network node incidence that takes 0 except for $\eta_{K+1,d} = 1$ and $\eta_{0,o} = -1$

Acknowledgments

This work was supported by JSPS KAKENHI Grants 24K00999.

Appendix A. List of notation

Table 1 lists notations frequently used in this paper.

Appendix B. Proof of Proposition 3.2

Proof. We apply Lemma 4.2 (and Proposition 4.3) of Akamatsu and Oyama (2024) to each of [SO/Sub-Shipper(j)] and [SO/Sub-Driver(t, w)] and obtain the objective value functions as follows:

$$z_j^*(\mathbf{y}) \approx \sum_{t \in \tilde{\mathcal{T}}} C_{j,t}^S y_{j,t} - \hat{\mathcal{H}}_j^S(\mathbf{y}_j) \quad \forall j \in \mathcal{J} \quad (\text{B.1})$$

$$z_{(t,w)}^*(\mathbf{f}) \approx \sum_{r \in \mathcal{R}} C_{(t,w),r}^D f_{(t,w),r} - \hat{H}_{(t,w)}^D(\mathbf{f}_{(t,w)}) \quad \forall (t, w) \in \mathcal{T} \times \mathcal{W} \quad (\text{B.2})$$

By summing up these over all tasks $j \in \mathcal{J}$ and all OD and time-window pairs $(t, w) \in \mathcal{T} \times \mathcal{W}$, we obtain the fluidly-approximated master problem [SO/Master-A]. $\square$

Appendix C. Proof of Proposition 3.3

Proof. The Lagrangian function $\mathcal{L}$ of [SO/Master-A] is:

$$\begin{aligned} \mathcal{L}(\mathbf{y}, \mathbf{f}, \lambda, \mu, \rho) = & \sum_{j \in \mathcal{J}} \left(\sum_{i \in \mathcal{T}} y_{j,i} (C_{j,i}^S - \mu_j + \lambda_{(i,j)}) - \hat{H}_j^S(\mathbf{y}_j) \right) \\ & + \sum_{i \in \mathcal{T}} \sum_{w \in \mathcal{W}} \left(\sum_{r \in \mathcal{R}} f_{(i,w),r} (C_{(i,w),r}^D - \sum_{j \in \mathcal{J}} \delta_{r,j} \lambda_{(i,j)} - \rho_{(i,w)}) - \hat{H}_{(i,w)}^D(\mathbf{f}_{(i,w)}) \right) \\ & + \sum_{j \in \mathcal{J}} \mu_j N_j^S + \sum_{i \in \mathcal{T}} \sum_{w \in \mathcal{W}} \rho_{(i,w)} N_{(i,w)}^D \end{aligned} \quad (\text{C.1})$$

where λ, μ, ρ are the Lagrangian multipliers with respect to (3.4), (3.7), (3.8), respectively.

From the optimality conditions with respect to $\mathbf{y}, \mathbf{f}, \lambda$, we obtain:

$$\begin{cases} y_{j,i}^* (C_{j,i}^S - \mu_j + \omega_{(i,j)} - \hat{H}_{j,i}^S) = 0, y_{j,i}^* \geq 0, C_{j,i}^S - \mu_j + \lambda_{(i,j)} - \hat{H}_{j,i}^S \geq 0 \\ f_{r,(i,w)}^{(t,w)*} (C_{(i,w),r}^D - \sum_{j \in \mathcal{J}} \delta_{r,j} \lambda_{(i,j)} - \rho_{(i,w)} - \hat{H}_{(i,w)}^D) = 0, f_{r,(i,w)}^{(t,w)*} \geq 0, C_{(i,w),r}^D - \sum_{j \in \mathcal{J}} \delta_{r,j} \lambda_{(i,j)} - \rho_{(i,w)} - \hat{H}_{(i,w)}^D \geq 0 \\ \lambda_{(i,j)}^* (\sum_{w \in \mathcal{W}} \sum_{r \in \mathcal{R}} f_{(i,w),r}^* - y_{j,i}^*) = 0, \lambda_{(i,j)}^* \geq 0, \sum_{w \in \mathcal{W}} \sum_{r \in \mathcal{R}} f_{(i,w),r}^* - y_{j,i}^* \geq 0. \end{cases} \quad (\text{C.2})$$

The first and second conditions are the random/perturbed utility maximization principles of shippers and drivers, and the third is the market clearing condition. Thus, we can interpret μ_j and $\rho_{(i,w)}$ as the expected minimum costs Π_j^S and $\Pi_{(i,w)}^D$, and $\lambda_{(i,j)} = p_{(i,j)}$. Moreover, as the perturbed functions $\hat{H}_j^S$ and $\hat{H}_{(i,w)}^D$ are the convex conjugate of Π_j^S and $\Pi_{(i,w)}^D$,

$$N_j^S \Pi_j^S(\mathbf{p}) = \sum_{i \in \mathcal{T}} y_{j,i}^* (C_{j,i}^S + p_{(i,j)}) - \hat{H}_j^S(\mathbf{y}^*) \quad (\text{C.3})$$

$$N_{(i,w)}^D \Pi_{(i,w)}^D(\mathbf{p}) = \sum_{r \in \mathcal{R}} f_{r,(i,w)}^* (C_{(i,w),r}^D - \sum_{j \in \mathcal{J}} \delta_{r,j} p_{(i,j)}) - \hat{H}_{(i,w)}^D(\mathbf{f}^*) \quad (\text{C.4})$$

As a result, the Lagrangian dual problem is

$$\max_{\lambda, \mu, \rho} \min_{\mathbf{y}, \mathbf{f}} \mathcal{L}(\mathbf{y}, \mathbf{f}, \lambda, \mu, \rho) = \max_{\mathbf{p}} \mathcal{L}(\mathbf{y}^*, \mathbf{f}^*, \mathbf{p}) = \sum_{j \in \mathcal{J}} N_j^S \Pi_j^S(\mathbf{p}) + \sum_{i \in \mathcal{T}} \sum_{w \in \mathcal{W}} N_{(i,w)}^D \Pi_{(i,w)}^D(\mathbf{p}) \quad (\text{C.5})$$

Moreover, $\Pi_{(i,w)}^D$ is equivalent to $\pi_{(0,o)}^{(t,w)}$, the expected minimum cost from the source state $(0, o)$ to the target state $(K+1, d)$ in the task-chain network of Fig. 3. $\square$

Appendix D. Proof of Proposition 6.4

Proof. Under the payment (6.2), the utility that shipper b gains is

$$\begin{aligned} u_b(\mathbf{s}_b, \mathbf{s}_{-b}) &= - \sum_{i \in \mathcal{T}} n_i^{b*}(\mathbf{s}_b, \mathbf{s}_{-b}) c_i^b - p_b(\mathbf{s}_b, \mathbf{s}_{-b}) \\ &= \left(- \sum_{i \in \mathcal{T}} n_i^{b*}(\mathbf{s}_b, \mathbf{s}_{-b}) c_i^b + \sum_{b' \neq b} \sum_{i \in \mathcal{T}} n_i^{b'*}(\mathbf{s}_b, \mathbf{s}_{-b}) s_i^{b'} \right) - \max_{\mathbf{n}_{-b}} z_{j,-b}^S(\mathbf{n}_{-b} | \mathbf{s}_{-b}, \mathbf{y}) \end{aligned} \quad (\text{D.1})$$

The final term is independent of b , and shipper b would choose $\mathbf{s}_b$ to maximize

$$- \sum_{i \in \mathcal{T}} n_i^{b*}(\mathbf{s}_b, \mathbf{s}_{-b}) c_i^b + \sum_{b' \neq b} \sum_{i \in \mathcal{T}} n_i^{b'*}(\mathbf{s}_b, \mathbf{s}_{-b}) s_i^{b'}.$$

Considering the assignment rule, $n_i^{b*}(-\mathbf{c}_b, \mathbf{s}_{-b})$ is the matching pattern that maximizes this. Therefore, for any $\mathbf{s}_b \neq -\mathbf{c}_b$, the following holds:

$$u_b(-\mathbf{c}_b, \mathbf{s}_{-b}) \geq u_b(\mathbf{s}_b, \mathbf{s}_{-b}), \quad (\text{D.2})$$

that is, the optimal behavior for b to maximize their utility is to announce the true valuation $\mathbf{s}_b = -\mathbf{c}_b$, which results in maximizing the social surplus. Thus, [VCG/Shipper] is *truthful* and *socially efficient* (see also e.g., Cramton et al., 2006). $\square$

Appendix E. Proof of Proposition 6.5

Proof. Under the reward scheme (6.9), the utility of driver a is

$$\begin{aligned}
u_a(\tilde{\mathbf{c}}_a, \tilde{\mathbf{c}}_{-a}) &= w_a(\tilde{\mathbf{c}}_a, \tilde{\mathbf{c}}_{-a}) - \sum_{k=0}^K \sum_{ij \in \mathcal{E}} m_{k,ij}^{a*}(\tilde{\mathbf{c}}_a, \tilde{\mathbf{c}}_{-a}) c_{ij}^a \\
&= \min_{\mathbf{m}_a} z_{-a}^D(\mathbf{m}_{-a} | \tilde{\mathbf{c}}_{-a}, \mathbf{x}) - \left(\sum_{k=0}^K \sum_{ij \in \mathcal{E}} m_{k,ij}^{a*}(\tilde{\mathbf{c}}_a, \tilde{\mathbf{c}}_{-a}) c_{ij}^a + \sum_{a' \neq a} \sum_{k=0}^K \sum_{ij \in \mathcal{E}} m_{k,ij}^{a'*}(\tilde{\mathbf{c}}_a, \tilde{\mathbf{c}}_{-a}) \tilde{c}_{ij}^{a'} \right).
\end{aligned} \tag{E.1}$$

Because the first term of the RHS is independent of a , driver a would choose bid $\tilde{\mathbf{c}}_a$ to minimize

$$\sum_{k=0}^K \sum_{ij \in \mathcal{E}} m_{k,ij}^{a*}(\tilde{\mathbf{c}}_a, \tilde{\mathbf{c}}_{-a}) c_{ij}^a + \sum_{a' \neq a} \sum_{k=0}^K \sum_{ij \in \mathcal{E}} m_{k,ij}^{a'*}(\tilde{\mathbf{c}}_a, \tilde{\mathbf{c}}_{-a}) \tilde{c}_{ij}^{a'}$$

Considering the assignment rule of [VCG/Driver] (in Step 2), $m_{k,ij}^{a'*}(\mathbf{c}_a, \tilde{\mathbf{c}}_{-a})$ would minimize this. Therefore, for any $\tilde{\mathbf{c}}_a \neq \mathbf{c}_a$, the following holds:

$$u_a(\mathbf{c}_a, \tilde{\mathbf{c}}_{-a}) \geq u_a(\tilde{\mathbf{c}}_a, \tilde{\mathbf{c}}_{-a}), \tag{E.2}$$

that is, announcing true perceived cost $\mathbf{c}_a$ by bidding is a (weakly) dominant strategy for each driver a , and in this case, a socially optimal matching is achieved. This proves that [VCG/Driver] is *truthful* and *efficient*. $\square$

Appendix F. Edge-based formulation of [SO]

Due to the difficulty of enumerating task bundles, in the numerical experiments of Section 8 we solve [SO] using an edge-based formulation on the task-chain network, as follows.

$$\begin{aligned}
[\text{SO-Edge}] \quad \min_{\mathbf{m}, \mathbf{n}} \quad z(\mathbf{m}, \mathbf{n}) &\equiv \sum_{j \in J} \left(\sum_{b \in B_j} \sum_{t \in \tilde{T}} n_t^b c_t^b \right) + \sum_{t \in \tilde{T}} \sum_{w \in \mathcal{W}} \left(\sum_{a \in \mathcal{A}_{(t,w)}} \sum_{k=0}^K \sum_{ij \in \mathcal{E}} m_{k,ij}^a c_{ij}^a \right)
\end{aligned} \tag{F.1}$$

$$\begin{aligned}
\text{subject to} \quad \sum_{t \in \tilde{T}} n_t^b &= 1 \quad \forall b \in B \quad (2.10) \\
n_t^b &\in \{0, 1\} \quad \forall t \in \tilde{T}, \forall b \in B \quad (2.11)
\end{aligned}$$

$$\sum_{i \in \tilde{J}} m_{k-1,ij}^a - \sum_{l \in \tilde{J}} m_{k,lj}^a = \eta_{k,j} \quad \forall k \in \{0, \dots, K+1\}, \forall j \in J, \forall a \in \mathcal{A} \tag{F.2}$$

$$m_{k,ij}^a \in \{0, 1\} \quad \forall k \in \{0, \dots, K+1\}, \forall ij \in \mathcal{E}, \forall a \in \mathcal{A} \tag{F.3}$$

$$\sum_{b \in B_j} n_t^b \leq \sum_{w \in \mathcal{W}} \sum_{a \in \mathcal{A}_{(t,w)}} \sum_{k=0}^K \sum_{ij \in \mathcal{E}} m_{k,ij}^a \quad \forall j \in J, \forall t \in \tilde{T} \tag{F.4}$$

Data availability

The data and code of this study are available at the author's GitHub repository (<https://github.com/yuki-oyama/csd-elas>).

References

- Akamatsu, T., 1996. Cyclic flows, Markov process and stochastic traffic assignment. *Transp. Res. Part B: Methodol.* 30 (5), 369–386.
- Akamatsu, T., 1997. Decomposition of path choice entropy in general transport networks. *Transp. Sci.* 31 (4), 349–362.
- Akamatsu, T., Oyama, Y., 2024. A fluid-particle decomposition approach to matching market design for crowdsourced delivery systems. *Transp. Res. Part C: Emerg. Technol.* 166, 104738.
- Akamatsu, T., Satsukawa, K., Oyama, Y., 2023. Global stability of day-to-day dynamics for schedule-based Markovian transit assignment with boarding queues. *arXiv preprint arXiv:2304.02194*.
- Allahviranloo, M., Baghestani, A., 2019. A dynamic crowdshipping model and daily travel behavior. *Transp. Res. Part E: Logist. Transp. Rev.* 128, 175–190.
- Alnaggar, A., Gzara, F., Bookbinder, J.H., 2021. Crowdsourced delivery: A review of platforms and academic literature. *Omega* 98, 102139.
- Archetti, C., Savelsbergh, M., Speranza, M.G., 2016. The vehicle routing problem with occasional drivers. *European J. Oper. Res.* 254 (2), 472–480.
- Arslan, A.M., Agatz, N., Kroon, L., Zuidwijk, R., 2019. Crowdsourced delivery—a dynamic pickup and delivery problem with ad hoc drivers. *Transp. Sci.* 53 (1), 222–235.
- Baillon, J.B., Cominetti, R., 2008. Markovian traffic equilibrium. *Math. Program.* 111 (1–2), 33–56.
- Barbosa, M., Pedroso, J.P., Viana, A., 2023. A data-driven compensation scheme for last-mile delivery with crowdsourcing. *Comput. Oper. Res.* 150, 106059.
- Beck, A., Teboulle, M., 2009. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM J. Imaging Sci.* 2 (1), 183–202.
- Bell, M.G.H., 1995. Alternatives to Dial's logit assignment algorithm. *Transp. Res. Part B* 29 (4), 287–295.
- Çınar, A.B., Dullaert, W., Leitner, M., Paradiso, R., Waldherr, S., 2024. The role of individual compensation and acceptance decisions in crowdsourced delivery. *Transp. Res. Part C: Emerg. Technol.* 169, 104834.
- Clarke, E.H., 1971. Multipart pricing of public goods. *Public Choice* 11, 17–33.
- Cramton, P., Shoham, Y., Steinberg, R., 2006. *Combinatorial Auctions*. The MIT Press.
- Dahle, L., Andersson, H., Christiansen, M., Speranza, M.G., 2019. The pickup and delivery problem with time windows and occasional drivers. *Comput. Oper. Res.* 109, 122–133.
- Dayarian, I., Savelsbergh, M., 2020. Crowdshipping and same-day delivery: Employing in-store customers to deliver online orders. *Prod. Oper. Manage.* 29 (9), 2153–2174.

- Dial, R.B., 1971. A probabilistic multipath traffic assignment algorithm which obviates path enumeration. *Transp. Res.* 5 (2), 83–111.
- Du, M., Zhou, J., Li, G., Tan, H., Chen, A., 2024. A stochastic ridesharing user equilibrium model with origin-destination-based ride-matching strategy. *Transp. Res. Part E: Logist. Transp. Rev.* 189, 103688.
- Elsokkary, N., Otrók, H., Singh, S., Mizouni, R., Barada, H., Omar, M., 2023. Crowdsourced last mile delivery: Collaborative workforce assignment. *Internet Things* 22, 100692.
- Fosgerau, M., Frejinger, E., Karlstrom, A., 2013. A link based network route choice model with unrestricted choice set. *Transp. Res. Part B: Methodol.* 56, 70–80.
- Fosgerau, M., Melo, E., De Palma, A., Shum, M., 2020. Discrete choice and rational inattention: A general equivalence result. *Internat. Econom. Rev.* 61 (4), 1569–1589.
- Fosgerau, M., Paulsen, M., Rasmussen, T.K., 2022. A perturbed utility route choice model. *Transp. Res. Part C: Emerg. Technol.* 136, 103514.
- Fudenberg, D., Iijima, R., Strzalecki, T., 2015. Stochastic choice and revealed perturbed utility. *Econometrica* 83 (6), 2371–2409.
- Furuhata, M., Dessouky, M., Ordóñez, F., Brunet, M.E., Wang, X., Koenig, S., 2013. Ridesharing: The state-of-the-art and future directions. *Transp. Res. Part B: Methodol.* 57, 28–46.
- Gansterer, M., Hartl, R.F., 2018. Centralized bundle generation in auction-based collaborative transportation. *Or Spectr.* 40 (3), 613–635.
- Groves, T., 1973. Incentives in teams. *Econometrica: J. Econ. Soc.* 41 (4), 617–631.
- Hou, S., Gao, J., Wang, C., 2022. Optimization framework for crowd-sourced delivery services with the consideration of shippers' acceptance uncertainties. *IEEE Trans. Intell. Transp. Syst.* 24 (1), 684–693.
- Hou, S., Wang, C., Gao, J., 2025. Reinforced stable matching for Crowd-Sourced Delivery Systems under stochastic driver acceptance behavior. *Transp. Res. Part C: Emerg. Technol.* 170, 104916.
- Huang, B., Zhu, H., Liu, D., Wu, N., Qiao, Y., Jiang, Q., 2021. Solving last-mile logistics problem in spatiotemporal crowdsourcing via role awareness with adaptive clustering. *IEEE Trans. Comput. Soc. Syst.* 8 (3), 668–681.
- Kafle, N., Zou, B., Lin, J., 2017. Design and modeling of a crowdsourcing-enabled system for urban parcel relay and delivery. *Transp. Res. Part B: Methodol.* 99, 62–82.
- Le, T.V., Stathopoulos, A., Van Woensel, T., Ukkusuri, S.V., 2019. Supply, demand, operations, and management of crowd-shipping services: A review and empirical evidence. *Transp. Res. Part C: Emerg. Technol.* 103, 83–103.
- Le, T.V., Ukkusuri, S.V., 2019a. Influencing factors that determine the usage of the crowd-shipping services. *Transp. Res. Rec.* 2673 (7), 550–566.
- Le, T.V., Ukkusuri, S.V., 2019b. Modeling the willingness to work as crowd-shippers and travel time tolerance in emerging logistics services. *Travel. Behav. Soc.* 15, 123–132.
- Le, T.V., Ukkusuri, S.V., Xue, J., Van Woensel, T., 2021. Designing pricing and compensation schemes by integrating matching and routing models for crowd-shipping systems. *Transp. Res. Part E: Logist. Transp. Rev.* 149, 102209.
- Mancini, S., Gansterer, M., 2022. Bundle generation for last-mile delivery with occasional drivers. *Omega* 108, 102582.
- Meng, Q., Lam, W.H., Yang, L., 2008. General stochastic user equilibrium traffic assignment problem with link capacity constraints. *J. Adv. Transp.* 42 (4), 429–465.
- Mladenow, A., Bauer, C., Strauss, C., 2016. "Crowd logistics": the contribution of social crowds in logistics activities. *Int. J. Web Inf. Syst.* 12, 379–396.
- Nesterov, Y., 1983. A method of solving a convex programming problem with convergence rate $O(1/k^2)$. *Sov. Math. Dokl.* 27 (2), 372–376.
- Nie, Y., Zhang, H., Lee, D.H., 2004. Models and algorithms for the traffic assignment problem with link capacity constraints. *Transp. Res. Part B: Methodol.* 38 (4), 285–312.
- Oyama, Y., 2023. Capturing positive network attributes during the estimation of recursive logit models: A prism-based approach. *Transp. Res. Part C: Emerg. Technol.* 147, 104014.
- Oyama, Y., Hara, Y., Akamatsu, T., 2022. Markovian traffic equilibrium assignment based on network generalized extreme value model. *Transp. Res. Part B: Methodol.* 155, 135–159.
- Oyama, Y., Hato, E., 2019. Prism-based path set restriction for solving Markovian traffic assignment problem. *Transp. Res. Part B: Methodol.* 122, 528–546.
- Paloheimo, H., Lettenmeier, M., Waris, H., 2016. Transport reduction by crowdsourced deliveries—a library case in Finland. *J. Clean. Prod.* 132, 240–251.
- Rockafellar, R.T., 1970. *Convex Analysis*. Princeton University Press.
- Simoni, M.D., Marcucci, E., Gatta, V., Claudel, C.G., 2020. Potential last-mile impacts of crowdshipping services: A simulation-based evaluation. *Transportation* 47, 1933–1954.
- Simoni, M.D., Winkenbach, M., 2023. Crowdsourced on-demand food delivery: An order batching and assignment algorithm. *Transp. Res. Part C: Emerg. Technol.* 149, 104055.
- Sørensen, J.R.V., Fosgerau, M., 2022. How McFadden met Rockafellar and learned to do more with less. *J. Math. Econom.* 100, 102629.
- Su, E., Qin, H., Li, J., Pan, K., 2023. An exact algorithm for the pickup and delivery problem with crowdsourced bids and transshipment. *Transp. Res. Part B: Methodol.* 177, 102831.
- Transportation Networks for Research Core Team, 2023. *Transportation networks for research*. URL: <https://github.com/bstabler/TransportationNetworks>. (Accessed 24 March 2023).
- Vickrey, W., 1961. Counterspeculation, auctions, and competitive sealed tenders. *J. Financ.* 16 (1), 8–37.
- Vohra, R.V., 2011. *Mechanism design: a linear programming approach*, Vol. 47. Cambridge University Press.
- Wang, L., Xu, M., Qin, H., 2023. Joint optimization of parcel allocation and crowd routing for crowdsourced last-mile delivery. *Transp. Res. Part B: Methodol.* 171, 111–135.
- Wang, Y., Zhang, D., Liu, Q., Shen, F., Lee, L.H., 2016. Towards enhancing the last-mile delivery: An effective crowd-tasking model with scalable solutions. *Transp. Res. Part E: Logist. Transp. Rev.* 93, 279–293.
- Yang, Y., Umboh, S.W., Ramezani, M., 2024. Freelance drivers with a decline choice: Dispatch menus in on-demand mobility services for assortment optimization. *Transp. Res. Part B: Methodol.* 190, 103082.
- Yao, R., Fosgerau, M., Paulsen, M., Rasmussen, T.K., 2024. Perturbed utility stochastic traffic assignment. *Transp. Sci.* 58 (4), 876–895.